

2025秋 EE351 《微机原理与微系统》

EE351

Microcomputer and Microsystems

姜俊敏

电子与电子工程系

JUNMIN JIANG

DEPARTMENT OF ELECTRONIC AND ELECTRICAL
ENGINEERING

第二章 微型计算机基础知识

Chapter 2 Basic knowledge of microcomputer

- 本章学习目标:

Learning objectives of this chapter:

- 1. 掌握微型计算机中的数制及其编码

Master the number system and its code in microcomputer

- 2. 掌握布尔代数和常见逻辑电路

Master Boolean algebra and common logic circuits

- 3. 了解微型计算机的常用技术术语和技术

Understand common technical terms and techniques for microcomputers

第二章 微型计算机基础知识

Chapter 2 Basic knowledge of microcomputer

● 2.1 微型机中的数制及其编码

Number system and coding in microcomputer

- 2.1.1 数与数制
Numbers and Number Systems
- 2.1.2 不同数制之间的转换
Conversion between different number systems
- 2.1.3 数制数据的编码及其运算
Number data encoding and its operation
 - 1.原码 2.反码; 3.补码; 4.十进制数的编码((非)压缩BCD码)
1.true form; 2.inverse code; 3.two's complement; 4.Coding of decimal numbers (non-compression BCD code)
 - 2.1.4 非数值数据的编码 (ASCII码, 汉字编码)
Non-numeric data encoding (ASCII code, Chinese character encoding)

● 2.2 布尔代数和常见逻辑电路

Boolean algebra and common logic circuits

● 2.3 微型计算机的常用技术术语和技术

Common technical terms and techniques of microcomputer

- 2.3.1 常用单位及术语;
Common units and terms;
- 2.3.2 常见技术
Common techniques

2.1 微型机中的数制及其编码

Number system and coding in microcomputer

2.1.1 数与数制

Numbers and Number Systems

进位计数制，简称数制。system of numeration, abbreviated as number system.

十进制：人们习惯采用的计数制是十进制。

Decimal: The number system that people are used to is the decimal system.

0—9 十个不同的基数，逢十进一。

0-9 ten different bases, with a carry of 1 for every ten.

用D(Decimal)表示或省略。

Use D (Decimal) to indicate or omit.

二进制：计算机所采用的计数制是二进制。

Binary: The counting system used by computers is binary.

只有0、1两个不同的基数，逢二进一。

There are only two different bases, 0 and 1, and you carry over when you get to two.

用B(Binary)表示。 Using B (Binary).

因为计算机用晶体管截止、饱和两个态下的输出电平1、0表示数字。

Because the output level of a computer transistor in the cutoff and saturation states is 1 and 0 to represent a number.

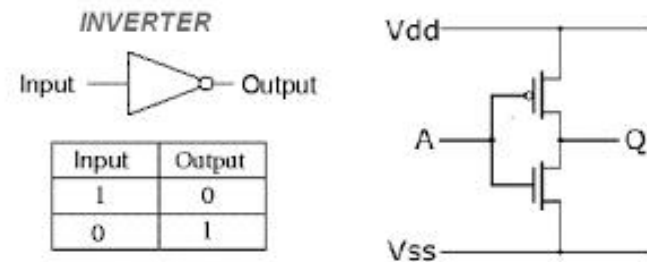
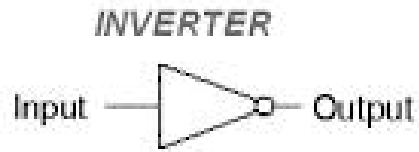


Fig. 1.1 Symbol, circuit structure and truth table of a CMOS inverter

2.1 微型机中的数制及其编码

Number system and coding in microcomputer



Input	Output
1	0
0	1

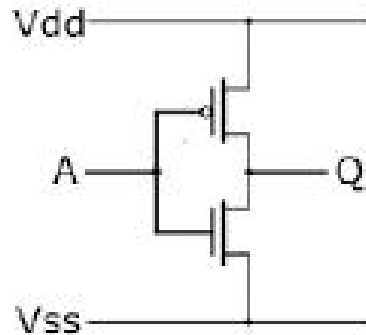
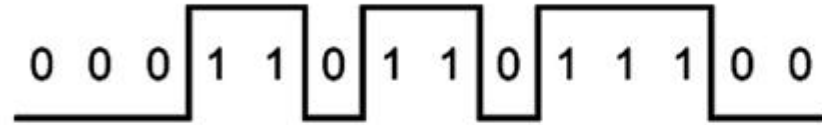
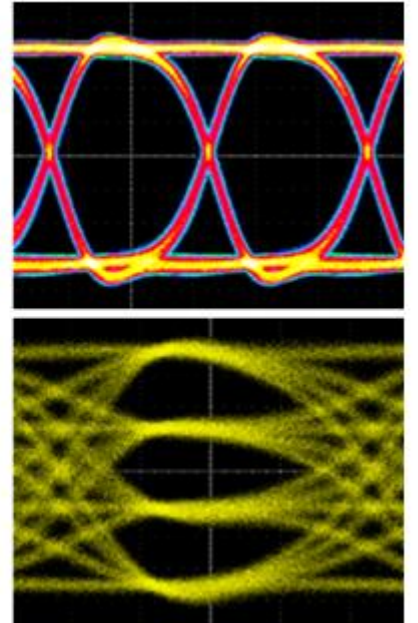
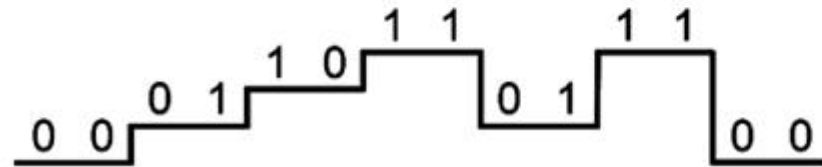


Fig. 1.1 Symbol, circuit structure and truth table of a CMOS inverter

PAM2-NRZ



PAM4



PAM (**P**ulse-**A**mplitude **M**odulation)

Binary: PAM 2

PMA 4: 00, 01, 11, 10 2bits

2. 1. 1 数与数制

Numbers and Number Systems

十进制与二进制转换

Conversion between decimal and binary

人机交互时采用十进制，计算机内数据存储、计算、处理用二进制，需要进行转换。在计算机中的解决方法是，利用接口技术作转换。

Human-computer interaction adopts decimal, and the data storage, calculation and processing in the computer are binary, which need to be converted. The solution in the computer is to use interface technology for conversion.

◇ 如：用键盘输入数据时使用十进制数，即输入电路使用的键盘是十进制数，输入接口电路将十进制数转换为二进制数后送到机器内部。

For example, when the data is input by the keyboard in decimal number, that is, the keyboard used by the input circuit is decimal number, and the input interface circuit converts the decimal number to binary number and sends it to the machine.

◇ 显示器显示十进制，输入显示器的数据为二进制/十六进制。

The display shows decimal and inputs data to the display in binary/hexadecimal.

2.1.2 不同数制之间的转换

Conversion between different number systems

1、十进制数转换为二进制数

Convert decimal numbers to binary numbers

十进制数转换为二进制数的方法

The method of converting a decimal number to a binary number

- 整数部分转换方法: "除2取余, 逆序排列"

Integers conversion method: "Divide by 2 and reverse order"

- 整数(及商)除以2取余, 直到商为0为止。最后将所有余数倒序排列, 得到转换结果。

The integer (and quotient) is divided by 2 and taken as a remainder until the quotient is 0. Finally, all the remainders are arranged in reverse order to get the conversion result.

- 小数部分转换方法: "乘2取整, 顺序排列"

The decimal conversion method is: "Multiply by 2 and take the integer, arrange the order."

- 乘以2取整, 直到满足精度要求为止。

Take the integer of 2 until the desired accuracy is achieved.

例1：将十进制数100转换为二进制数

Example 1: Convert the decimal number 100 to a binary number

	余数 remainder	低位 Low bit
2 100	0	↑ High bit
2 50	0	
2 25	1	
2 12	0	
2 6	0	
2 3	1	
2 1	1	
0		

整数部分转换方法：
除2取余，逆序排列

Whole number conversion method:
Take the remainder modulo 2, then reverse the order.

$(100)_{10} = (01100100)_2$

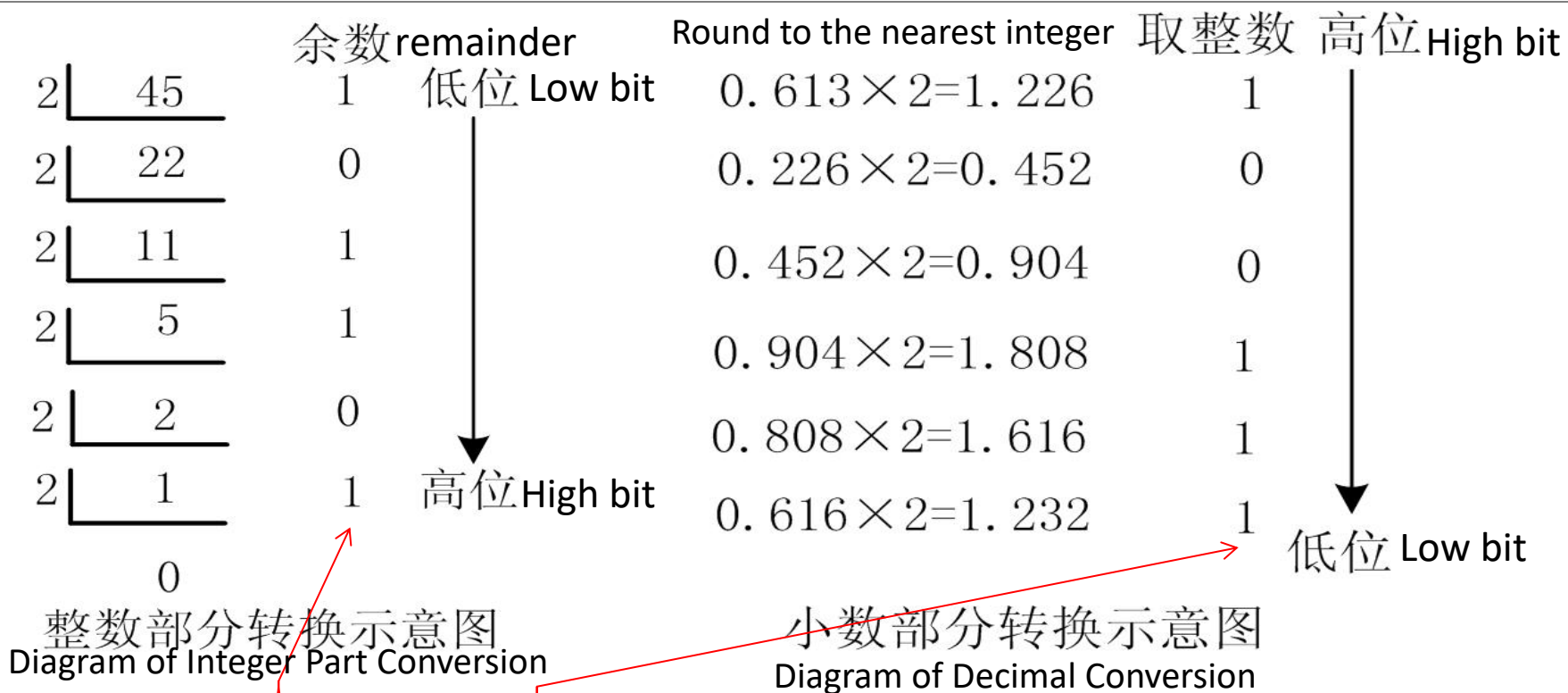
或者表示为：

Or expressed as:

$100D = 01100100B$

例2：将十进制数45.613转换成二进制数

Example 2: Convert the decimal number 45.613 to binary



$(45.613)_{10} \approx (101101.100111)_2$ 或 $45.613D \approx 101101.100111B$

整数部分转换：除2取余，逆序排列

小数部分转换：乘2取整，顺序排列

Whole number conversion: divide by 2 and take the remainder, then reverse the order

Conversion of decimal part: multiply by 2 and take the integer, in order

2、二进制数转换为十进制数

Convert binary numbers to decimal numbers

转换方法：按权展开相加。

Conversion method: expand and add according to the power.

例如，一个8位二进制数11111111B的各位的权值依次是 2^7 、 2^6 、 2^5 、...、 2^0 。
For example, the weights of the bits in an 8-bit binary number 1111 1111B are 2^7 、 2^6 、 2^5 、...、 2^0 , respectively.

如将10110110B转换为十进制数的方法是：
The method of converting 1011 0110B to a decimal number is:

$$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 182$$

即， $(10110110)_2 = (182)_{10}$

That is,

或表示为：10110110B=182D=182

or expressed as:

3、十六进制数

Hexadecimal numbers

优点：二进制数位数较多时，读写不方便。
而使用十六进制表示简明。

Advantages: It is not convenient to read and write when the number of binary digits is large. However, hexadecimal is concise.

十六进制数表示：

0~9、A、B、C、D、E、F十六个不同的基数

0~9, A, B, C, D, E, F sixteen different bases

逢十六进一，用H（Hexadecimal）表示。

In case of sixteen bits, use H (Hexadecimal).

3、十六进制数

Hexadecimal numbers

◇ 二进制数转换为十六进制数

Binary number to hexadecimal number

原理：四位二进制数对应一位十六进制，所以二进制整数转换为16进制时，从最低位开始，每四位一组（不足四位时高位补0）转换成一位十六进制数据即可。例如：1011 0110B=B6H; 1011 0110B=B6H。

◇ Binary number to hexadecimal number

Principle: Four binary numbers correspond to one hexadecimal number. Therefore, when a binary integer is converted to hexadecimal, it is converted into one hexadecimal data from the lowest bit, with each four groups (if less than four bits, 0 is added to the higher bit).

For example: 1011 0110B=B6H; 1011 0110B=B6H

十六进制数转换为二进制数

Hex number to binary number

十六进制数据转换为二进制数据时，把每一位十六进制数据直接写成四位二进制数。

When hexadecimal data is converted to binary data, each hexadecimal data bit is written directly as a four-bit binary number.

例如：64H=0110 0100B For example: 64H=0110 0100B

表2-1 4位二进制数和一位十六进制数具有一一对应关系

Table 2-1 One-to-one correspondence between 4-bit binary numbers and one hexadecimal number

十六进制 hexadecimal	二进制 binary	十六进制 hexadecimal	二进制 binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

十六进制数据和十进制数据之间的转换

Conversion between hexadecimal data and decimal data

- 通过二进制进行相互转换;

Interconvert between binary;

- 十进制数转换成十六进制:

Converting decimal numbers to hexadecimal:

将十进制整数除以十六取余，小数部分乘以十六取整，可直接转换为十六进制数；

Divide a decimal integer by 16 and take the remainder, multiply the decimal part by 16 and take the integer, which can be directly converted into a hexadecimal number;

- 十六进制数转换成十进制:

Converting hexadecimal numbers to decimal:

将十六进制数按权展开相加得到十进制数。

Add the hexadecimal numbers in power form to get the decimal number.

$n+1$ 位十六进制数的权值分别为 16^n 、 16^{n-1} 、...、 16^2 、 16^1 、 16^0

The weights of the $n+1$ hexadecimal numbers are 16^n 、 16^{n-1} 、...、 16^2 、 16^1 、 16^0

2.1.3 数制数据的编码及其运算

Number data encoding and its operation

在计算机中，数据分**无符号数**和**带符号数**。

In a computer, data is divided into unsigned and signed numbers.

无符号数用整个**机器字长**的全部二进制位表示数值位, 无符号位;

An unsigned number uses all the binary bits of the entire machine word to represent the value, an unsigned bit;

带符号数用**最高位**表示该数的**符号位**。

The highest bit is used to represent the sign of the number.

8位: $(100)_{10} = (01100100)_2$
8-bit:

8位字长: $(-100)_{10} = (11100100)_2$ **8位字长: $(+100)_{10} = (01100100)_2$**
8-bit word length: 8-bit word length:

带符号数又有**原码**、**补码**和**反码**三种形式。

The number is represented in three forms: true form, two's complement code and inverse code.

➤ **本小节介绍: 1. 原码; 2. 反码; 3. 补码; 4. 十进制数的编码 ((非) 压缩BCD码)**

This section introduces: 1. True form; 2. Inverse code; 3. Two's complement; 4. Coding of decimal numbers

1、原码表示法

True form representation

由于计算机中只能有0、1两种数，不仅数的数值部分在计算机中用0、1编码的形式表示，正、负号也只能用0、1编码表示。

Because there are only 0 and 1 numbers in a computer, not only the numerical part of the number is represented by 0 and 1 codes in the computer, but also the positive and negative signs can only be represented by 0 and 1 codes.

一般用数的最高位(Most Significant Bit, MSB)表示数的正负符号。

The most significant bit (MSB) of a number generally indicates the positive or negative sign of the number.

例如，若用5位二进制数表示数据时，最高位表示符号，0表示正数，1表示负数，余下的四位表示数据：

For example, if a 5-bit binary number is used to represent data, the highest bit represents the sign, 0 for a positive number and 1 for a negative number, and the remaining four bits represent the data:

MSB=0表示正数，如+1011B表示为01011B；

MSB=0 represents a positive number, such as + 1011 B is represented as 0 1011 B;

MSB=1表示负数，如-1011B表示为11011B。

MSB=1 represents a negative number, such as -1011 B is represented as 1 1011 B.

机器数和真值

Number of machines and truth values

- 把一个数在机器内的二进制形式称为机器数。

The binary form of a number in a machine is called a machine number.

- 把这个数本身称为该机器数的真值。

Call this number itself the true value of the machine number.

- 前面“01011B”和“11011B”就是两个机器数。它们的真值分别为+1011B和-1011B。

The previous "0 1011B" and "1 1011 B" are two machine numbers. Their true values are + 1011 B and -1011 B respectively.

- 真值为纯小数时，其原码形式为 $X_s.X_1X_2\cdots X_n$ ，其中 X_s 表示符号位。例如，

When the true value is a pure decimal, its true form form is $X_s.X_1X_2\cdots X_n$, where X_s represents the sign bit. For example,

- 若 $X=+0.0110B$ ，则 $[X]_{\text{原}}=0.0110B$ ；若 $X=-0.0110B$ ，则 $[X]_{\text{原}}=1.0110B$

If $X = + 0.0110 B$, then $[X]_{\text{original}} = 0.0110 B$; If $X = -0.0110 B$, then $[X]_{\text{original}} = 1.0110 B$

- 真值为纯整数时，其原码形式为 $X_sX_nX_{n-1}\cdots X_2X_1$ ，其中 X_s 表示符号位。例 $[X]_{\text{原}}=00110B$ ； $[X]_{\text{原}}=10110B$

- When the true value is a pure integer, its true form form is $X S X_n X_{n-1} \dots X_2 X_1$, where $X S$ represents the sign bit. Example: $[X]_{\text{original}} = 0 0110 B$; $[X]_{\text{original}} = 1 0110 B$

1、原码表示法

true form representation

8位二进制原码的表示范围为 $-2^7-1 \sim +2^7-1$:

The representation range of 8-bit binary true form is $-2^7-1 \sim +2^7-1$:

$-127 \sim -0 \sim +0 \sim +127$

16位二进制原码的表示范围为 $-2^{15}-1 \sim +2^{15}-1$:

The representation range of 16-bit binary true form is $-2^{15}-1 \sim +2^{15}-1$:

$-32767 \sim -0 \sim +0 \sim +32767$

原码表示中, 真值0有两种不同的表示形式(8位字长):

In the true form representation, there are two different representations for true value 0 (8-bit word length):

$[+0]_{\text{原}} = 0000\ 0000\text{B}$, $[-0]_{\text{原}} = 1000\ 0000\text{B}$

无符号数

unsigned number

没有符号位的数，称为无符号数。

A number without a sign is called an unsigned number.

如字长为8位时，能表示的无符号数的最大值为11111111B，即255(2^8-1)，而8位有符号数的最大值是01111111B，即+127(2^7-1)。

For example, when the word length is 8 bits, the maximum value of the unsigned number that can be represented is 11111111B, that is, 255(2^8-1), while the maximum value of the 8-bit signed number is 01111111B, that is, +127(2^7-1).

无符号数的最大值比有符号数大一倍。

The maximum value of an unsigned number is twice as large as that of a signed number.

8位二进制无符号数的表示范围为：0~255 (2^8-1)

The representation range of 8-bit unsigned binary numbers is: 0~255 (2^8-1)

16位二进制无符号数的表示范围为：0~65535 ($2^{16}-1$)

The representation range of a 16-bit unsigned binary number is 0 to 65535 ($2^{16}-1$)

表2-2 8位二进制数的原码、反码、补码的表示

Table 2-2 Representation of true form, inverse code and two's complement code of 8-bit binary number

无符号数 unsigned number		有符号数 signed number			
十进制数 decimal number	二进制数 binary number	真值 truth value	原码 true form	反码 Inverse code	补码 complement code
127	0111 1111B	+127	0111 1111B	0111 1111B	0111 1111 B
...	...	:	:	:	:
1	0000 0001B	+1	0000 0001B	0000 0001B	0000 0001 B
0	0000 0000B	+0	0000 0000B	0000 0000 B	0000 0000 B
128	1000 0000B	-0	1000 0000B	1111 1111 B	0000 0000 B
129	1000 0001B	-1	1000 0001B	1111 1110 B	1111 1111 B
...	...	:	:	:	:
255	1111 1111B	-127	1111 1111B	1000 0000 B	1000 0001 B
		-128	不能表示	不能表示	1000 0000 B

可见，8位字长原码、反码表示的数的范围: +127~−127，而补码表示的数的范围: +127~−128。

It can be seen that the range of numbers represented by 8-bit original code and inverse code is +127~ -127, while the range of numbers represented by complement code is +127~ -128.

Quiz

- 1、8位二进制原码的表示范围是 ()
The representation range of the true form 8-bit binary code is ()
- 2、8位二进制反码的表示范围是 ()
The representation range of 8-bit binary inverse code is ()
- 3、8位二进制补码的表示范围是 ()
The range of the 8-bit binary complement is ()
- 4、-0的8位二进制补码为 ()
The 8-bit binary complement of -0 is ()

Quiz

- 1、8位二进制原码的表示范围是 (-127 ~ +127)
The representation range of the true form 8-bit binary code is (-127 ~ +127)
- 2、8位二进制反码的表示范围是 (-127 ~ + 127)
The representation range of 8-bit binary inverse code is (-127 ~ +127)
- 3、8位二进制补码的表示范围是 (-128 ~ +127)
The range of the 8-bit binary complement is (-128 ~ +127)
- 4、-0的8位二进制补码为 (0000 0000B)
The 8-bit binary complement of -0 is (0000 0000B)

(4) Arithmetic operations on numerical data

➤ ① overflow

When the result of adding or subtracting unsigned numbers exceeds the number's range, it is called **carry or borrow**. Computers have dedicated flag bits to record this information. As long as these flags are handled appropriately, the result will not be erroneous. Therefore, carry and borrow handling must be considered during addition and subtraction operations involving multi-byte numbers.

Program Status Word (PSW) Bit Definitions

bit	D7	D6	D5	D4	D3	D2	D1	D0
symbol	CY	AC	F0	RS1	RS0	OV	F1	P

CY: Carry flag

OV: Overflow flag

(4) Arithmetic operations on numerical data

◇ Key Considerations When Performing Addition and Subtraction Operations Using Two's Complement:

➤②After adopting two's complement representation, **if a carry occurs during signed arithmetic operations, this carry is discarded** without affecting the result. The sign of the result is determined by the sign of the operands (**provided no overflow occurs**).

➤③Properties of Complement Operations:

$$[X]_{\text{补}} + [Y]_{\text{补}} = [X + Y]_{\text{补}}$$

$$[X]_{\text{补}} - [Y]_{\text{补}} = [X - Y]_{\text{补}}$$

➤The above properties of operations are independent of the number of digits n.

Example:

$$\begin{array}{rcl} [X]_{\text{补}} & = & 00101101 \quad (45D) \\ + [Y]_{\text{补}} & = & 11111111 \quad (-1D) \\ \hline [X]_{\text{补}} + [Y]_{\text{补}} & = & 100101100 \quad (44D) \end{array}$$

Arithmetic operations on numerical data example

The result cannot exceed the range of numbers it can represent; otherwise, an overflow error will occur.

Using two's complement arithmetic in binary numbers as an example, illustrate the operational properties of signed numbers.

【Example 2-2】 It is known : $X = +0101101B(45D)$, $Y = -0000001B$,
Solve for $X + Y = ?$ (Subtracting two positive numbers: 45 minus 1)

$$[X]_{\text{补}} + [Y]_{\text{补}} = [X+Y]_{\text{补}}$$

$$[X+Y]_{\text{原}} = [[X+Y]_{\text{补}}]_{\text{补}} = [[X]_{\text{补}} + [Y]_{\text{补}}]_{\text{补}} = 00101100B$$

Solution:

$$\begin{array}{r} [X]_{\text{补}} = 00101101 \quad (45D) \\ + [Y]_{\text{补}} = 11111111 \quad (-1D) \\ \hline [X]_{\text{补}} + [Y]_{\text{补}} = 100101100 \quad (44D) \end{array}$$

→ Discard the carry

◇ Subtraction as addition: Using two's complement, the operation of subtracting a positive number can be transformed into the addition operation of adding the two's complement of that negative number.

Arithmetic operations on numerical data

example

【Example 2-3】 It is known that: $X = -0001101B$,

$Y = -0000001B$, solve for: $X + Y = ?$

Solution: $[X]_{\text{补}} = 11110011$

$+ [Y]_{\text{补}} = 11111111$

$[X]_{\text{补}} + [Y]_{\text{补}} = 111110010$

 Discard the carry

$$[X]_{\text{补}} + [Y]_{\text{补}} = [X+Y]_{\text{补}} = [-0001110B]_{\text{补}} = 11110010B$$

So , $[X+Y]_{\text{原}} = [-0001110B]_{\text{原}} = 10001110B = [[X+Y]_{\text{补}}]_{\text{补}}$

$$X+Y = -0001110B$$

$[[X+Y]_{\text{补}}]_{\text{补}} = [X+Y]_{\text{原}}$ always holds true

Arithmetic operations on numerical data example

【Example2-4】 It is known that : $X = +1$, $Y = -128$

Solve for: $X + Y = ?$

Solution: $[X]_{\text{补}} = 00000001$

$+ [Y]_{\text{补}} = 10000000$

$[X + Y]_{\text{补}} = 10000001$

$[X]_{\text{补}} + [Y]_{\text{补}} =$

Because $[[X + Y]_{\text{补}}]_{\text{补}} = [X + Y]_{\text{原}} = 1111\ 1111\text{B} = [-127]_{\text{原}}$

So , $X + Y = -111\ 1111 = -127$

Strictly speaking, this question $[[X + Y]_{\text{补}}]_{\text{补}} = X + Y$ is not valid,
Nevertheless $[[X+Y]_{\text{补}}]_{\text{补}} = [X+Y]_{\text{原}}$ always holds true,

An adder can perform all arithmetic operations

Addition becomes subtraction: Since subtracting a positive number can be viewed as adding a negative number, in computing, after obtaining the two's complement, the operation of subtracting a positive number is transformed into adding the two's complement of that negative number.

Addition as Multiplication: This can be accomplished using the method of shifting and adding.

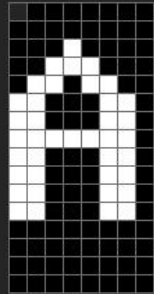
Addition as Division: This is achieved using the method of shifting and subtracting. This approach enables all arithmetic operations to be performed using only an adder.

2.1.4 Encoding of Non-Numeric Data

Computers can process not only numerical data but also **text and other non-numerical data information**.

Non-numeric data refers to information that cannot undergo arithmetic operations, such as **characters**, strings, graphic symbols, **Chinese characters**, speech, images, and other forms of data.

When transmitting this information, **the original values are not directly transmitted or processed**. Instead, they **undergo specific processing according to certain rules to adopt a universal transmission format**. The resulting processed information is termed **encode**.

英文字模	位代码	字模信息
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 1 0 0 0 0	0x10
	0 0 1 1 1 0 0 0	0x38
	0 1 1 0 1 1 0 0	0x6c
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 1 1 1 1 1 0	0xfe
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00

(A's ASCII Code 41h)

A's bitmap font

Calls bitmap or vector fonts for display or printing
Display or print using bitmap or vector fonts

1、ASCII encoding

When processing text files, each character is composed of its corresponding standard character matrix (e.g., the matrix for the character A). The text file itself does not contain these matrices but uses their encoding (A's ASCII Code 41h) to represent each character.

For example, when editing Chinese text using zone-based encoding, a 4-digit decimal zone code can represent ten thousand distinct characters.

The internationally recognized standard character encoding is ASCII (American Standard Code for Information Interchange).

Appendix A provides the ASCII Code table.

(A的ASCII码41h)

英文字模	位代码	字模信息
	0 0 0 0 0 0 0 0 0 0	0x00
	0 0 0 1 0 0 0 0 0 0	0x10
	0 0 1 1 1 0 0 0 0 0	0x38
	0 1 1 0 1 1 0 0 0 0	0x6c
	1 1 1 0 0 0 1 1 0 0	0xc6
	1 1 0 0 0 1 1 0 0 0	0xc6
	1 1 1 1 1 1 1 1 0 0	0xfe
	1 1 0 0 0 1 1 0 0 0	0xc6
	1 1 0 0 0 1 1 0 0 0	0xc6
	1 1 0 0 0 1 1 0 0 0	0xc6
	1 1 0 0 0 1 1 0 0 0	0xc6
	0 0 0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0 0 0	0x00

ASCII encoding

ASCII表																										
(American Standard Code for Information Interchange 美国标准信息交换代码)																										
高四位 低四位		ASCII控制字符											ASCII打印字符													
		0000					0001						0010	0011	0100	0101	0110	0111								
		0					1						2	3	4	5	6	7								
		十进制	字符	Ctrl	代码	转义 字符	字符解释	十进制	字符	Ctrl	代码	转义 字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0	空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH		标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☹	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX		正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK		肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	•	^G	BEL	\a	响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	▢	^H	BS	\b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	HT	\t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◉	^J	LF	\n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	\v	纵向制表	27	←	^[	ESC	\e	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	\f	换页	28	└	^_	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	♪	^M	CR	\r	回车	29	↔	^]	GS		组分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO		移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	🎵	^O	SI		移入	31	▼	^-	US		单元分隔符	47	/	63	?	79	O	95		111	o	127	␣	^Backspace 代码: DEL

A's ASCII Code 41h, or 65

1、ASCII编码

ASCII表																									
(American Standard Code for Information Interchange 美国标准信息交换代码)																									
高四位 低四位	ASCII控制字符												ASCII打印字符												
	0000						0001						0010		0011		0100		0101		0110		0111		
	0						1						2		3		4		5		6		7		
	十进制	字符	Ctrl	代码	转义	字符解释	十进制	字符	Ctrl	代码	转义	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0 空字符	16	▶	^P	DLE	数据链路转义	32		48	0	64	@	80	P	96	`	112	p		
0001	1	1	☺	^A	SOH	标题开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q		
0010	2	2	☹	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r		
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s		
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t		
0101	5	5	♣	^E	ENQ	查询	21	§	^U	NAK	否定应答	37	%	53	5	69	E	85	U	101	e	117	u		
0110	6	6	♠	^F	ACK	肯定应答	22	—	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v		
0111	7	7	•	^G	BEL	\a 响铃	23	↕	^W	ETB	传输块结束	39	'	55	7	71	G	87	W	103	g	119	w		
1000	8	8	▣	^H	BS	\b 退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x		
1001	9	9	○	^I	HT	\t 横向制表	25	↓	^Y	EM	介质结束	41	)	57	9	73	I	89	Y	105	i	121	y		
1010	A	10	◼	^J	LF	\n 换行	26	→	^Z	SUB	替代	42	*	58	:	74	J	90	Z	106	j	122	z		
1011	B	11	♂	^K	VT	\v 纵向制表	27	←	^[	ESC	\e 溢出	43	+	59	;	75	K	91	[	107	k	123	{		
1100	C	12	♀	^L	FF	\f 换页	28	└	^\	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124			
1101	D	13	♪	^M	CR	\r 回车	29	↔	^]	GS	组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}		
1110	E	14	🎵	^N	SO	\s 移出	30	▲	^^	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~		
1111	F	15	🎵	^O	SI	\s 移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣	^Backspace 代码: DEL	

附录 A ASCII 码表

ASCII 值	十六进制	控制字符	ASCII 值	十六进制	控制字符	ASCII 值	十六进制	控制字符	ASCII 值	十六进制	控制字符
0	0	NUL	32	20	(space)	64	40	@	96	60	'
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	\$	68	44	D	100	64	d
5	5	END	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

ASCII encoding

ASCII defines a total of 256 codes (from 0 to 255).

- 0 - 31 and 127(del) (33 in total) are control characters (ASCII control characters).
- 32 (space) - 126 are printable characters (ASCII printable characters).
- 0-127 constitutes the standard ASCII encoding.
- 128 - 255 represents the extended ASCII encoding, which is no longer an international standard.

◇ **Standard ASCII Code: 95 characters represented by 7-bit binary encoding**

26 uppercase letters: 41H–5AH

26 lowercase letters: 61H–7AH

10 numeric digits (0–9): 30H–39H

33 special characters: [, +, −, @, |, #, etc.

1、ASCII encoding

- Appendix A presents the ASCII character table, which represents character codes using 8-bit binary numbers. The fundamental code occupies 7 bits, with the 8th bit available for parity checking,
- By setting the parity bit to either “1” or “0,” the number of “1”s within an 8-bit byte is maintained as either odd (known as odd parity) or even (known as even parity). This is commonly used during the serial transmission of characters or numbers to detect errors that may occur during transmission.

汉字编码

(1) --汉字输入编码

汉字信息处理系统一般包括编码、输入、编辑、存储、输出和传输。编码是关键。汉字输入编码是用计算机标准键盘上按键的不同排列组合来对汉字进行编码。

Chinese character information processing systems typically consist of encoding, input, editing, storage, output, and transmission processes, with encoding being the critical component. Without proper encoding solutions, Chinese characters cannot be processed by computers. The input encoding system uses specific combinations of keystrokes on standard computer keyboards to encode these characters.

2、汉字编码

(1) --汉字输入编码

- ◇ 常用输入编码有数字、字音、字形和音形编码等。
 - ◆ 数字编码：如电报码、区位码等。无重码，但难记。
 - ◆ 字音编码：以汉语拼音作为编码基础。简单易学，但重码很高，如搜狗拼音、全拼、双拼等。
 - ◆ 字形编码：如五笔字型码、郑码等。有重码少的优点。
 - ◆ 音形编码：音形编码吸取了音码和形码的优点，使编码规则简化，重码少。常用的有全息码等。
- ◇ Common input codes include numeric, phonetic, glyph and phonographic codes.
 - ◆ Digital codes: such as telegraph code, location code, etc. No duplicate code, but difficult to remember.
 - ◆ Word-based encoding: Based on Pinyin as the basis of encoding. It is easy to learn, but the emphasis on code is very high, such as Sogou Pinyin, full Pinyin, double Pinyin and so on.
 - ◆ Character encoding: such as Wubi character code, Zheng code, etc. It has the advantage of less redundant codes.
 - ◆ Phonographic code: The phonographic code draws on the advantages of phonographic code and graphic code, simplifying the coding rules and reducing the number of codes. Common ones include holographic code.

(2) 汉字国标码

汉字国家标准编码即国标码，是不同汉字信息处理系统间进行汉字交换时所用的编码。以国家标准局GB2312-80规定的汉字交换码作为标准汉字编码。共收录7445个。

汉字区位码：在字符集中,汉字和字符分94个区,每区94位。每个汉字及字符用两个字节表示，前一字节为区码，后一字节为位码，各用两位16进制数字表示。

The National Standard Code for Chinese Characters (NSCC), also known as the Guobiao Code, is an encoding standard used for character exchange between different information processing systems. The GB2312-80 encoding standard, established by Chinas National Bureau of Standards, serves as the primary reference for character encoding and contains 7,445 standardized characters.

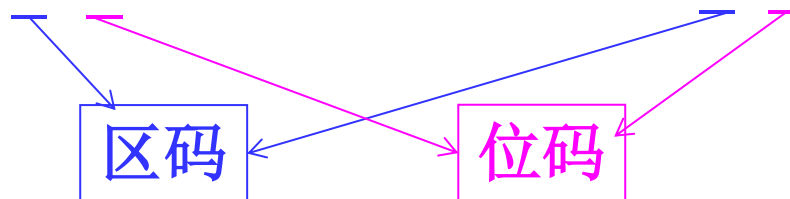
Chinese character location code: In the character set, Chinese characters and characters are divided into 94 zones, each of which is 94 bits. Each Chinese character and character is represented by two bytes, the first byte is the zone code, the second byte is the bit code, each of which is represented by two hexadecimal digits.

(2) 汉字国标码

汉字区位码不等于汉字国标码，两者间关系：

国标码 = 区位码（化成16进制） + 2020H

例："啊"的区位码为"1601"，用16进制表示为1001H，国标码为3021H。



The Chinese character location code is not equal to the Chinese character national standard code. The relationship between them:

National code = location code (converted to hexadecimal) + 2020H

For example, the location code of "ah" is "1601", which is expressed in hexadecimal as 1001H and the national standard code is 3021H.

(3) 汉字机内码

汉字机内码简称汉字内码,是在计算机外部设备和信息系统内部存储、处理、传输汉字用的代码。

在西文计算机中,无交换码和内码之分,一般以ASCII码作为内码。英文字符的机内码是7位ASCII码,最高位为0 (即D7=0)。

汉字内码用两个字节表示: 为区分汉字与英文字符,将汉字国标码每字节的最高位置1,即汉字机内码。

The Chinese character internal code, abbreviated as the Chinese character internal code, is a code used to store, process and transmit Chinese characters in the internal storage, processing and transmission of computer external devices and information systems.

In western computers, there is no exchange code and internal code, generally ASCII code as the internal code. The internal code of English characters is 7-bit ASCII code, the highest bit is 0 (that is, D7 = 0).

The Chinese character internal code is expressed in two bytes: to distinguish Chinese characters from English characters, the highest bit of each byte of the Chinese character national standard code is 1, that is, the Chinese character

(3) 汉字机内码

如, “啊”的国标码为00110000 00100001 (3021H),

机内码为10110000 10100001 (B0A1H)

汉字机内码 = 汉字国标码 + 8080H

For example, the national code for "ah" is 0 011 0000 0
010 0001 (3021H),

The internal code is 1 011 0000 1 010 0001 (B0A1H)

Chinese character machine code = Chinese character
national standard code + 8080H

(4) 汉字字形码

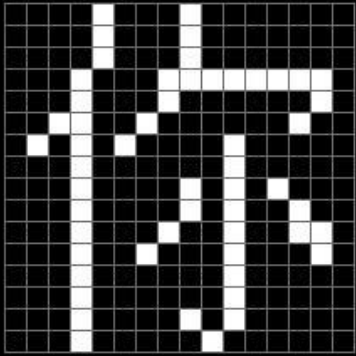
汉字用点阵方式表示其外形，这个点阵称为**汉字字模**，也称为**汉字字形码**。

不管汉字笔划多少，都可在同样的方块中书写，把方块分割为许多小方块，组成一个**点阵**，每个小方块就是点阵中的一点，即二进制的一位。每个点由**1和0**表示“白”和“黑”两种颜色。用这样的点阵就可输出汉字。

Chinese characters are represented by a dot matrix, which is called the Chinese character pattern, also called the Chinese character shape code.

16 *16点阵

16 *16 dot matrix

中文字模	位代码	字模信息
	0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0	0x08, 0x80
	0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0	0x08, 0x80
	0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0	0x08, 0x80
	0 0 0 1 0 0 0 1 1 1 1 1 1 1 1 0	0x11, 0xfe
	0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 0	0x11, 0x02
	0 0 1 1 0 0 1 0 0 0 0 0 0 0 1 0 0	0x32, 0x04
	0 1 0 1 0 1 0 0 0 0 1 0 0 0 0 0	0x54, 0x20
	0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0	0x10, 0x20
	0 0 0 1 0 0 0 0 1 0 1 0 1 0 0 0	0x10, 0xa8
	0 0 0 1 0 0 0 0 1 0 1 0 0 1 0 0	0x10, 0xa4
	0 0 0 1 0 0 0 1 0 0 1 0 0 1 1 0	0x11, 0x26
	0 0 0 1 0 0 1 0 0 0 1 0 0 0 1 0	0x12, 0x22
	0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0	0x10, 0x20
	0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0	0x10, 0x20
	0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0	0x10, 0xa0
	0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0	0x10, 0x40

“你”字的区位码是: 3667;
国标码: 4463H; 机内码: C4E3H

The zone code for the character “你” is: 3667;
GB code: 4463H; Internal code: C4E3H

(5) 各种代码间的逻辑关系

不同的输入编码
输入到计算机中，
都统一使用国标
码

Different
input codes
are uniformly
used in the
computer.

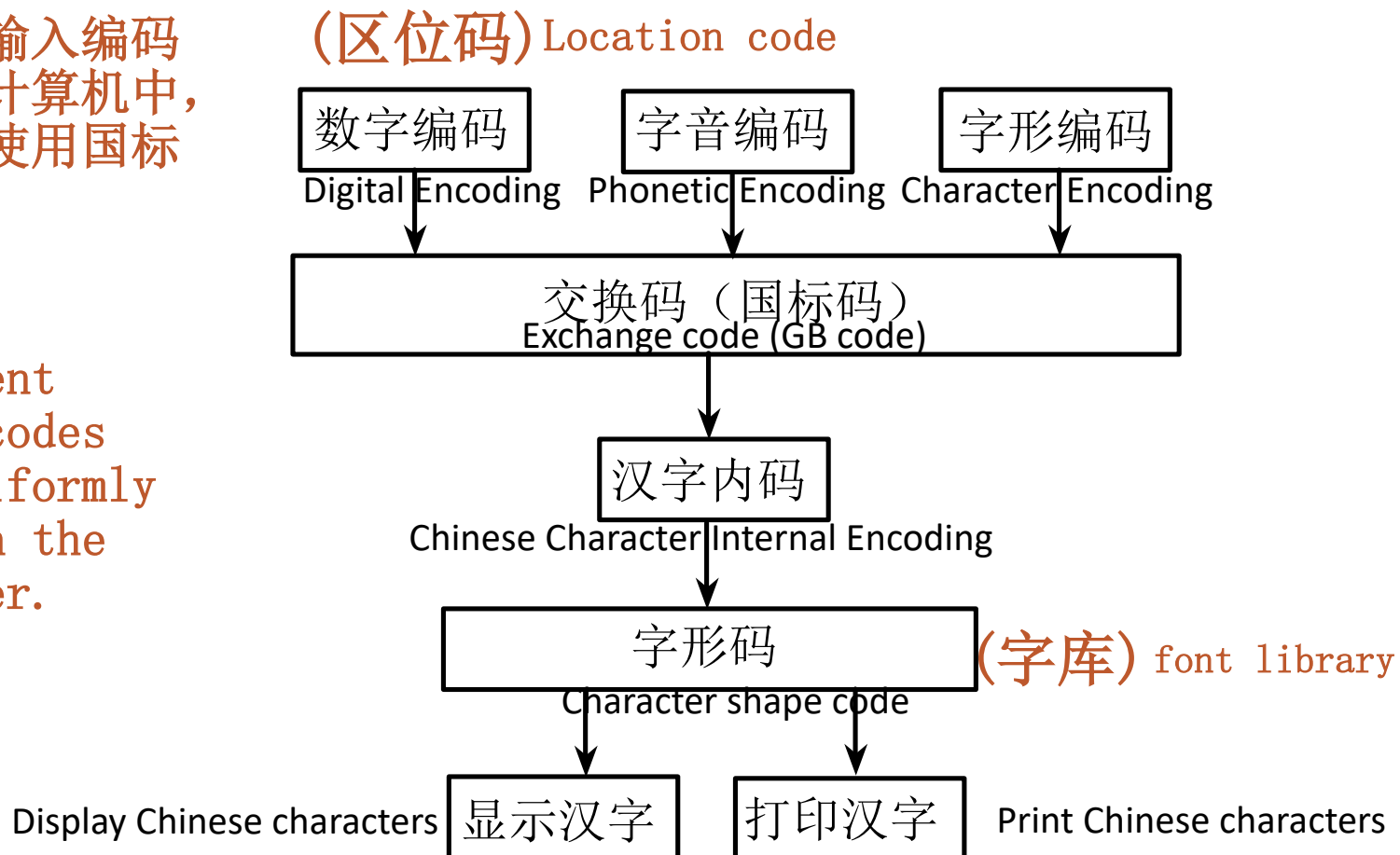


图2-5 各种代码间的逻辑关系

Figure 2-5 Logical Relationships Among Various Codes

汉字编码 举例

[例2-5] “春”的区位码为20-26, 计算其国标码和机内码。

区位码: 第1字节 第2字节

十进制 20 26

↓ ↓

十六进制 14H 1AH
+ 20H + 20H

国标码: 34H 3AH
+ 80H + 80H

机内码: B4H BAH

[Example 2-5] The location code of "spring" is 20-26. Calculate its national code and internal code.

Location code: First byte Second byte

Decade 20 26

↓ ↓

Hexadecimal 14H 1AH

+ 20H + 20H

National code: 34H 3AH

+ 80H + 80H

Internal code: B4H BAH

(6) 统一代码 Unicode

统一代码（Unicode）是一种全新的编码方法，此法有足够的 ability 来表示全世界多达6 800种语言中任意一种语言里使用的所有符号。

Unicode (Unified Code) is a new encoding method that is capable of representing all the symbols used in any of the worlds 6,800 languages.

其基本方法是，用1个16位的数来表示Unicode中的每个符号，即允许表示65 536个不同的字符或符号。这种符号集被称为基本多语言平面（BMP）。

The basic method is to use a 16-bit number to represent each symbol in Unicode, that is, to allow the representation of 65,536 different characters or symbols. This set of symbols is called the Basic Multilingual Plane (BMP).

2.2 布尔代数和常见逻辑电路

2.2.1 布尔代数

布尔代数是英国数学家乔治·布尔（George Boole）发明的，布尔也是数理逻辑的创始人。布尔代数在电子计算机的逻辑设计等工程技术领域中有重要的应用。

Boolean algebra was invented by British mathematician George Boole, who is also the founder of mathematical logic. Boolean algebra has important applications in engineering fields such as logical design of electronic computers.

◇ 布尔代数以命题为对象，包含三种基本逻辑操作（与，或，非）的完整代数学，它可对命题进行运算。

◇ 基本的运算符有 $+$ ， $\cdot$ 和 $'$ 。

二元运算 $+$ 称为布尔加法，布尔和，布尔并等；

二元运算 $\cdot$ 称为布尔乘法，布尔积，布尔交等；

一元运算 $'$ 称为布尔补，有时使用“ $\overline{}$ ”表示。

◇ Boolean algebra is a complete algebra that takes propositions as its objects and includes three basic logical operations (AND, OR, NOT). It can perform operations on propositions.

◇ The basic operation symbols are $+$, $\cdot$, and $'$. The binary operation $+$ is called Boolean addition, Boolean sum, or Boolean union, etc.;

The binary operation $\cdot$ is called Boolean multiplication, Boolean product, or Boolean intersection, etc.;

The unary operation $'$ is called Boolean complement, sometimes denoted by “ $\overline{}$ ”

运算的基本依据是下面的基本公式和规则：

The fundamental basis for operations is the following basic formulas and rules.

◦ 交换律： $A + B = B + A$

Commutative Law $A \cdot B = B \cdot A$

◦ 结合律： $A + (B + C) = (A + B) + C$
Combination Law

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

◦ 分配律： $A + (B \cdot C) = (A + B) \cdot (A + C)$
Distributive Law $A \cdot (B + C) = A \cdot B + A \cdot C$

◆ 吸收律： $A + A \cdot B = A$
Absorption Law $A \cdot (A + B) = A$

◆ 第二吸收律： $A + \overline{A} \cdot B = A + B$
The Second Absorption Law $A \cdot (\overline{A} + B) = A \cdot B$

运算的基本依据是下面的基本公式和规则:

The fundamental basis for operations is the following basic formulas and rules.

- 反演律(又称摩根定律): 常用于复杂逻辑函数的化简。

Inversion Law (also known as Morgan's Law): Often used to simplify complex logic functions.

$$\overline{A \cdot B \cdot C \cdot \dots} = \overline{A} + \overline{B} + \overline{C} + \dots$$

$$\overline{\overline{A \cdot B \cdot C \cdot \dots}} = \overline{A} + \overline{B} + \overline{C} + \dots$$

◆ 包含律:

Inclusion Law

$$AB + \overline{A}C + BC = AB + \overline{A}C$$

$$(A+B)(\overline{A}+C)(B+C) = (A+B)(\overline{A}+C)$$

◆ 重叠律: $A + A = A, \quad A \cdot A = A$

Overlap Law

◆ 互补律: $A + \overline{A} = 1, \quad A \cdot \overline{A} = 0$

Complementarity Law

◆ 0-1律: $0 + A = A, \quad 1 \cdot A = A, \quad 0 \cdot A = 0, \quad 1 + A = 1$

0-1 Law

2.2.2 基本逻辑运算和逻辑门电路

2.2.2. Basic Logic Operations and Logic Gate Circuits

- 逻辑运算主要包括与、或、非、异或等。逻辑变量只有两个：逻辑0和逻辑1。
- Logic operations mainly include AND, OR, NOT, XOR, etc.. Logic variables can only be logic 0 or logic 1.
- 逻辑运算只是按位进行运算，没有进位和借位关系，逻辑变量也没有符号问题。
- Logic operations are performed bit by bit, with no carry or borrow relationships, and logic variables do not have sign issues.。

◇ 逻辑与（AND）Logic AND

逻辑与也叫逻辑乘，其结果叫逻辑积

Logic AND is also called logical multiplication, and its result is the logical product.

运算规则：按位相与，当两逻辑变量都为逻辑1时，逻辑积才为1，否则逻辑积为0。

Operation rule: When performing a bitwise AND, the logical product is 1 only when both logic variables are 1; otherwise, the logical product is 0.

逻辑与的运算符号：“·”或“∧”。圆点也可省略

Symbol: "·" or "∧". The dot can also be omitted.

逻辑与（AND）Logic and (AND)

若用Y表示逻辑积，A和B分别表示两个逻辑变量时，其表达式为：

If Y represents the logical product and A and B are the two logic variables, the expression is:

$$Y = A \bullet B$$

A、B及其逻辑积Y的关系的真值表如表2-3所示。

The logic AND operation corresponds to the "AND gate" logic circuit.

For instance

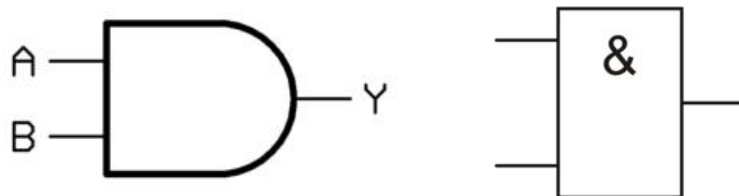
$$\begin{array}{r} 11010110 \\ \wedge \quad 00110101 \\ \hline 00010100 \end{array}$$

表2-3 逻辑与的真值表
Table 2-3: Logic AND Truth Table

A	B	Y=A·B
0	0	0
0	1	0
1	0	0
1	1	1

逻辑与运算对应逻辑电路“与门”。

Logic and operation correspond to the logic circuit "AND gate".



逻辑或（OR） Logic OR (OR)

逻辑或也叫**逻辑加**，其**结果叫逻辑和**。

运算的规则: **按位运算**，两逻辑变量中只要有一个为逻辑1时，逻辑或结果就为1，否则逻辑或结果为0。

逻辑或的运算符号: “+”或者“V”。

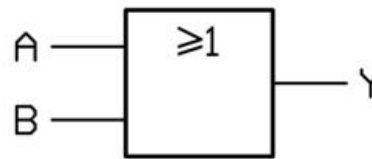
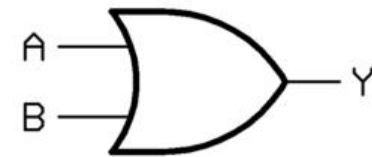
The logic is also called **logical addition**, and the result is called **logical addition**. Rule of operation: **bitwise operation**. In a logical or between two logical variables, the result is 1 if only one of them is 1, otherwise it is 0. Logical or operation symbol: "+" or "V".

◇若用Y表示逻辑和，A和B分别表示两个逻辑变量时，其表达式为：

◇If Y represents the logical sum and A and B are the two logic variables, the expression is:

$$Y = A + B$$

$$\begin{array}{r} 10010110 \\ \vee \quad 00110101 \\ \hline 10110111 \end{array}$$



OR gate

A	B	Y=A+B
0	0	0
0	1	1
1	0	1
1	1	1

Table 2-4: Logic OR Truth Table

逻辑非（NOT） Logic NOT (NOT)

逻辑非也叫逻辑反

Logic NOT is also called **logical inversion**.

运算规则：将一个变量按位求反的运算

Operation rule: It is a bitwise inversion of a variable.

Expression:

$$Y = \overline{A}$$

例：若A=01100011，则Y=A=10011100

Example: If A = 01100011, then Y = A = 10011100.

逻辑电路“非门”

The logic circuit is a **"NOT gate"**.

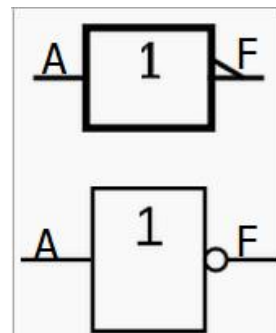
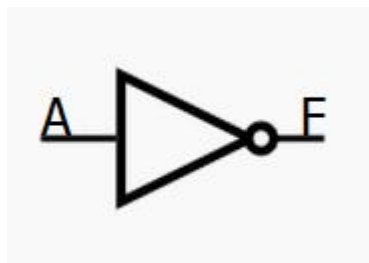


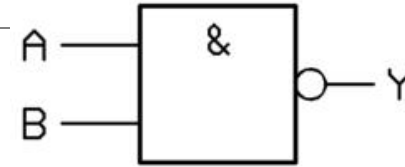
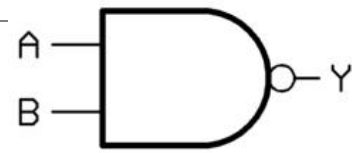
Table 2-5: Logic NOT Truth Table

A	$Y = \overline{A}$
0	1
1	0

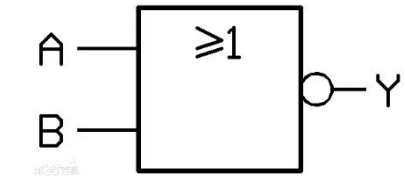
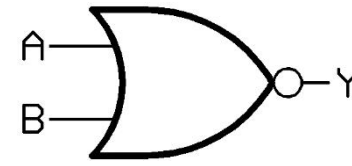
- 上述逻辑运算中，“非”运算的级别最高；“与”运算次之；“或”运算最低。
- In the logic operations above, **"NOT"** has the highest precedence, followed by **"AND"**, and **"OR"** has the lowest.

2、组合逻辑电路 2.Combinational Logic Circuits

与非门NAND Gate: $Y = \overline{A \bullet B}$



或非门NOR Gate: $Y = \overline{A + B}$

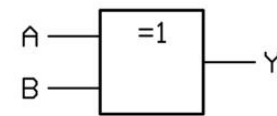
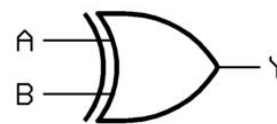


异或门XOR Gate: $Y = A \oplus B = A \bullet \bar{B} + \bar{A} \bullet B$

异或运算规则：两变量相同为0，不同为1。它可进行两逻辑变量不相等的逻辑测试。

XOR operation rule: The result is 0 if the two variables are the same, and 1 if they are different. It can be used to test for inequality between two logic variables.

Example:
$$\begin{array}{r} 10110110 \\ \oplus 00110101 \\ \hline 10000011 \end{array}$$



“异或门”XOR gate

Table 2-6: Logic XOR Truth Table

A	B	$Y=A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

2、组合逻辑电路 2.Combinational Logic Circuits

同或门 XNOR Gate (Coincidence Gate)

$$Y = A \odot B = A \cdot B + \bar{A} \cdot \bar{B}$$

逻辑同或真值表

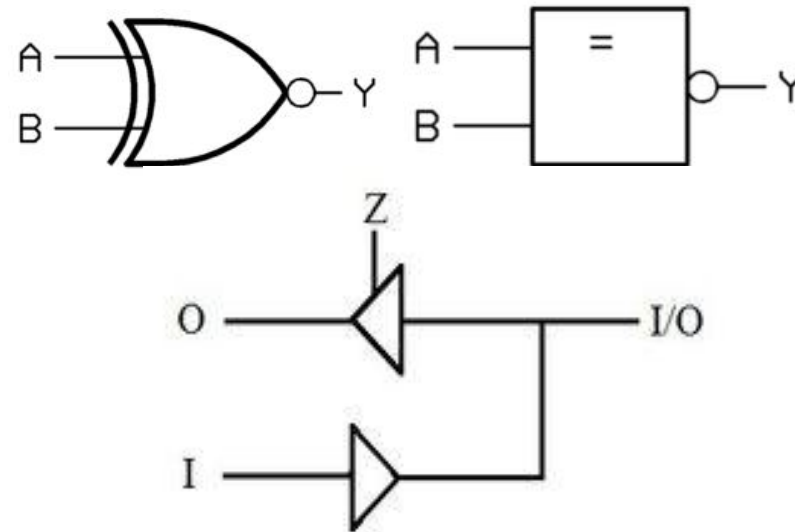
Logic XNOR Truth Table

A	B	$Y = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

三态门

在数字电路中，有些电路引脚输出状态除了高、低电平即1、0之外，还有第三种状态，即高阻状态，这种状态也叫做浮空、高阻、挂起等。具有高阻态输出的门电路称为三态门，常用于构成总线接收器和发送器。

“同或门” “XNOR gate”.



The application of the three states gate

◇Tri-state Gate

In digital circuits, some output pins can have a third state in addition to high and low levels (1 and 0), which is the high-impedance state. This state is also called floating, high-Z, or suspended. A gate circuit with a high-impedance output is called a tri-state gate and is often used to construct bus receivers and transmitters.

2、组合逻辑电路 2.Combinational Logic Circuits

Tri-state Buffer

Three-state Buffer, also known as a **tri-state gate** or **tri-state driver**, has an output controlled by an **enable input**

- When the enable input is **active**, the device outputs a normal logic state (Logic 0 or Logic 1).
- When the enable input is **inactive**, the output is in a **high-impedance (Hi-Z)** state, which is equivalent to being disconnected from the circuit.

Logic Symbols for Tri-state Buffers

Rectangular Symbols



Variant Symbols



High-Active Enable,
Non-inverting Output

Low-Active Enable,
Non-inverting Output

High-Active Enable,
Inverting Output

Low-Active Enable,
Inverting Output

Tri-state Gate

- 高阻状态是门电路输出端的上拉、下拉两个晶体管在控制端的作用下都处于截止状态，而截止状态所呈现的阻抗很高。
- The high-impedance state occurs when both the pull-up and pull-down transistors at the gate's output are in the off (cutoff) state, controlled by an enable input. The impedance in this state is very high
- 当输出端呈高阻状态时，输出端不输出任何逻辑电平，就像是开路一样。
- When the output is in a high-impedance state, it does not output any logic level, effectively acting like an open circuit.
- 每条线上不能同时有两个信号是输出状态，所以当几个芯片的输出端连在一起的时候，如其中一个是输出状态时，其他芯片的输出端不能同时也是输出状态，此时其他芯片既不能输出1，也不能输出0，就像是断开电路一样，这就是高阻状态，一般用符号Z来表示。
- When several chip outputs are connected, only one can be in an active output state at a time. The other chips must place their outputs in a high-impedance state so they don't drive the line to 1 or 0, which is like being disconnected from the circuit. This state is often represented by the symbol Z.

2.2.3 常用组合逻辑电路——1、锁存器

2.2.3 Common Combinational Logic Circuits——1. Latch

锁存: 把信号暂存以维持某种电平状态。

Latching: The process of temporarily storing a signal to maintain a specific level state.

74LS373:具有三态缓冲输出的8D锁存器，在单片机应用中，当地址线和数据线复用时，用作地址锁存器。

74LS373:An 8D-type latch with tri-state buffered outputs. In microcontroller applications, it is used as an address latch when address and data lines are multiplexed.

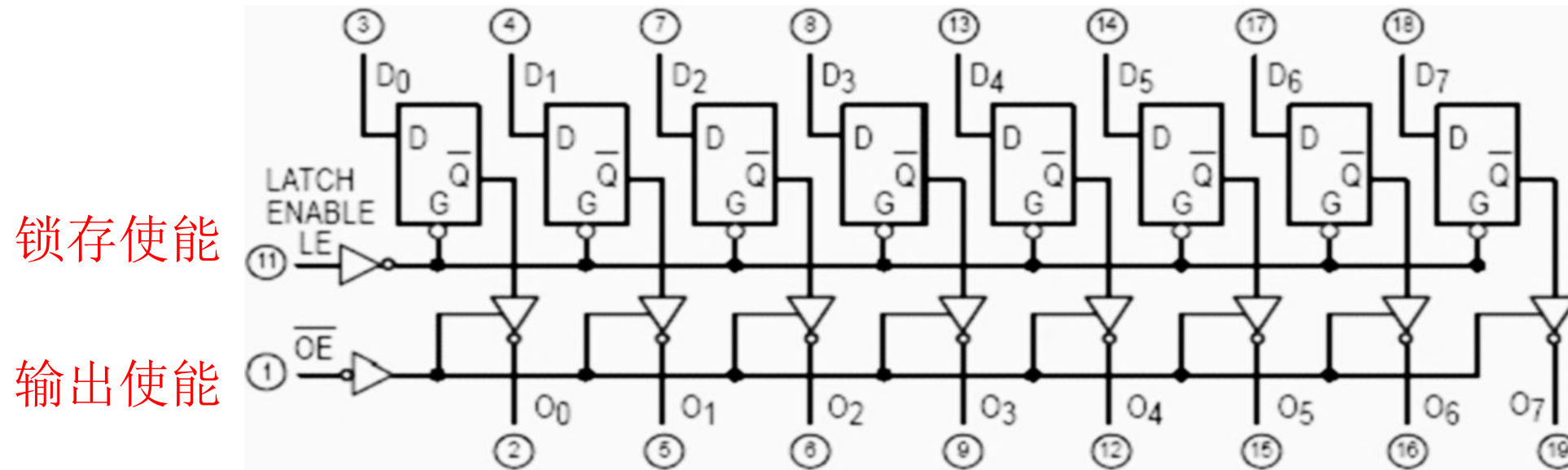


Figure 2-7: 74LS373 Internal Logic Diagram

74LS373的内部逻辑电路图如图2-7所示

The internal logic circuit diagram of 74LS373 is shown in Figure 2-7

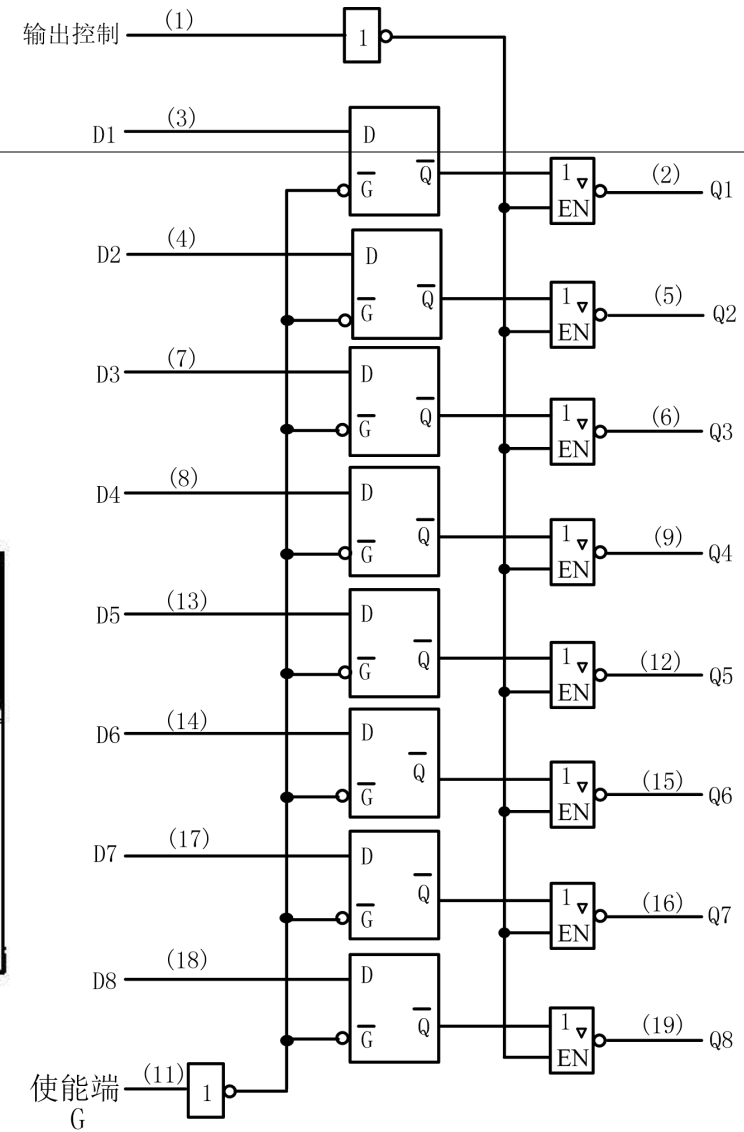
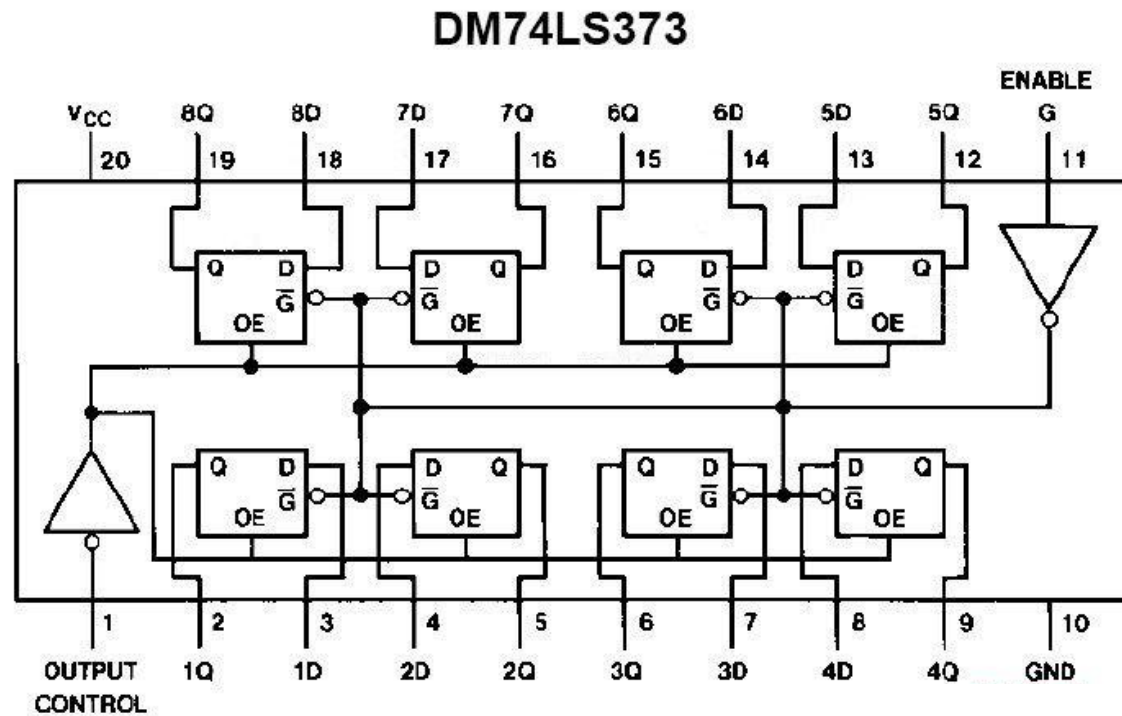


Figure 2-7: 74LS373 Internal Logic Diagram

74LS373引脚图 74LS373 Pinout

74LS373引脚图如图2-8所示。

- D0~D7 为数据输入端
- OC为输出允许控制端(三态，低电平有效)；
- G为锁存允许端(也称为LE端)；
- Q0~Q7为输出端。

74LS373 Pinout:

- D0~D7: Data input terminals.
- OC (Output Control): Output enable control terminal (tri-state, active low).
- G (Gate): Latch enable terminal (also called LE).
- Q0~Q7: Output terminals

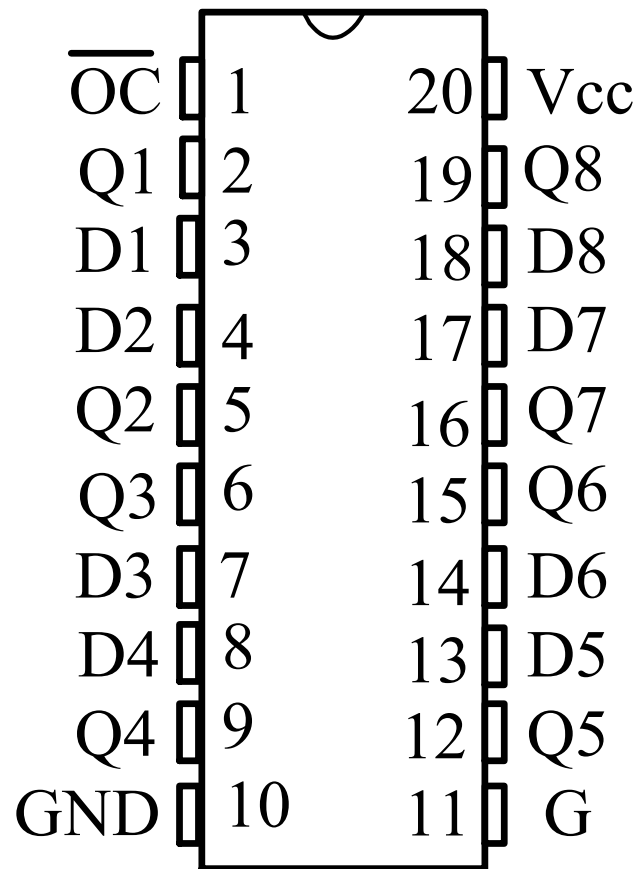
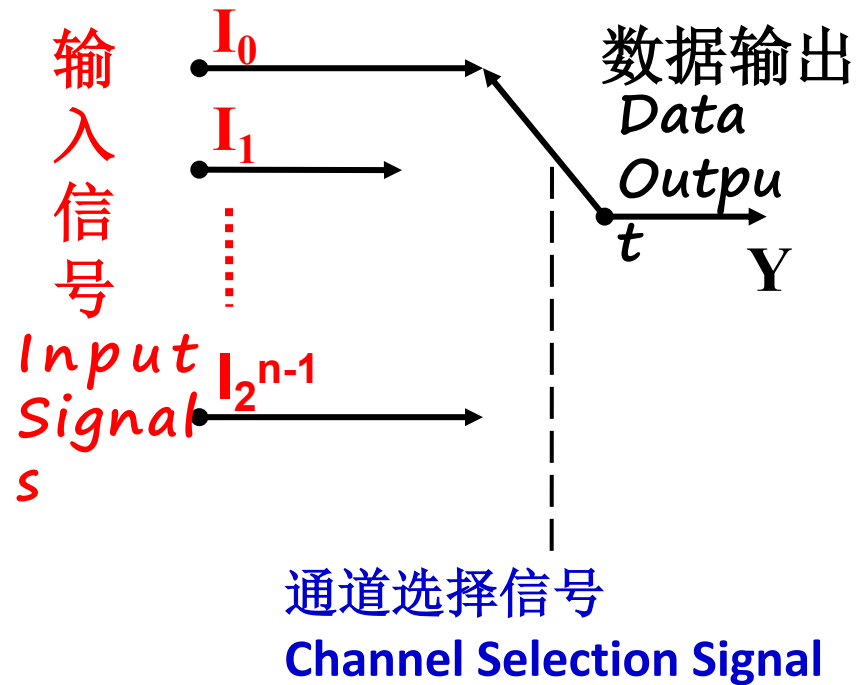


Figure 2-8: 74LS373 Pinout

2、数据选择器 2.Data Selector (Multiplexer)

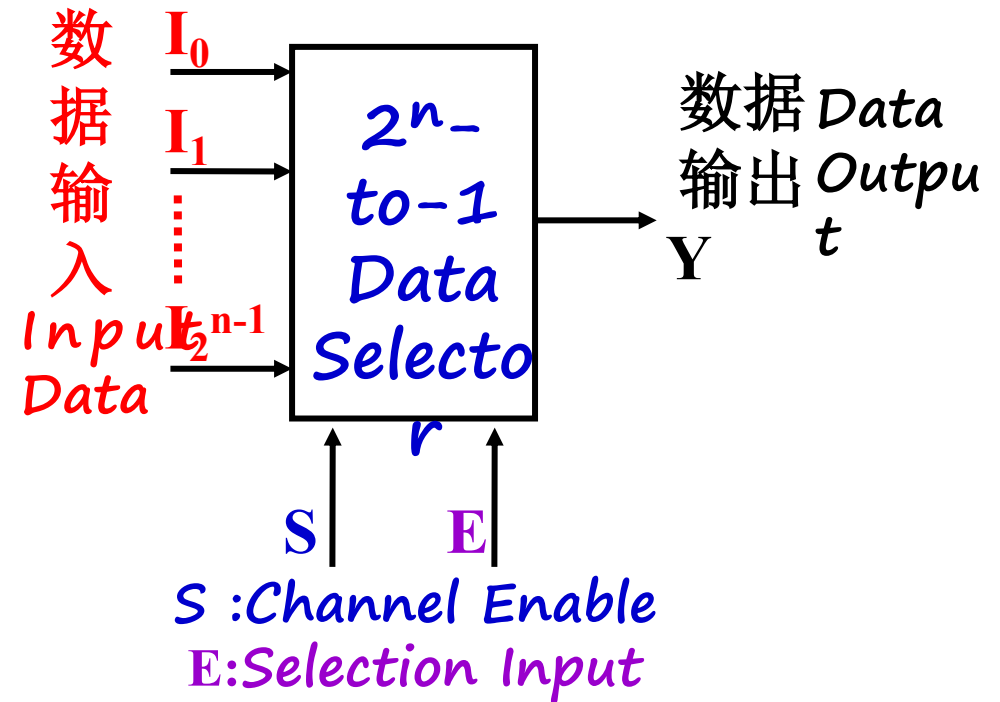
- 在数字电路中，常常需要把多个通道的信号传送到公共数据线上，完成这一功能的逻辑电路称为数据选择器。
- In digital circuits, it's often necessary to transmit signals from multiple channels onto a common data line. The logic circuit that performs this function is called a data selector.
- 数据选择器又称多路选择器或多路开关，是以“与或”门或“与或非”门为主体的电路，其功能是在通道选择信号的作用下，从多个数据输入通路中选择某一个通道的数据作为该选择器的输出。数据选择器的原理图如图2-9所示。
- A data selector, also known as a multiplexer or MUX, is a circuit primarily composed of AND-OR or AND-OR-INVERT gates. Its function is to select data from one of several input channels to be the output, based on a channel selection signal.

2.Data Selector (Multiplexer)



2^n -to-1 Data Selector

(a) Schematic Diagram



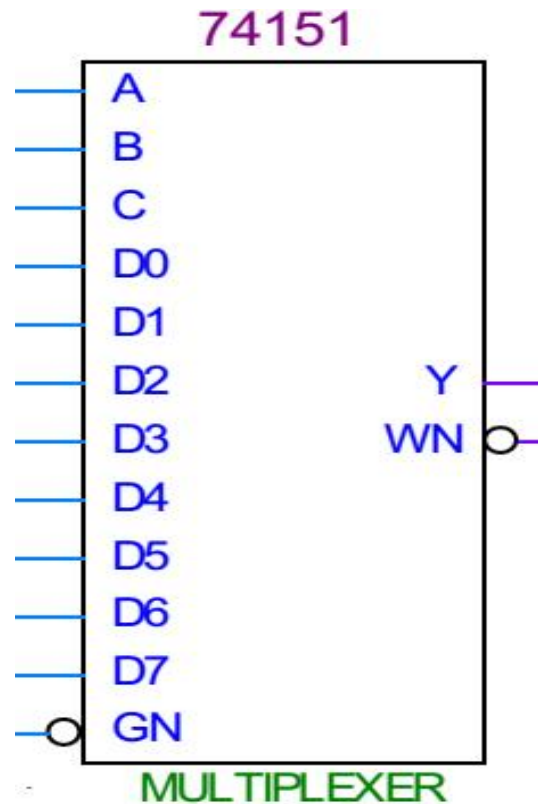
(b) Logic Diagram

Figure 2-9: Data Selector Principle Diagram

2.Data Selector (Multiplexer)

常见的8选1数据选择器有74151、74251；16选1数据选择器有74150，也可以用两片74151连接起来构成。

Common chips include the 74151 and 74251 (8-to-1 data selectors) and the 74150 (16-to-1 data selector). Two 74151 chips can also be combined to create a 16-to-1 selector.



3、译码器 3.Decoder

- ◇ 计算机的三总线是数据总线、地址总线和控制总线。
- ◇ The three buses of a computer are the data bus, the address bus and the control bus.
- ◇ 当计算机向一个芯片送出一个数据时，还要通过地址总线送出与这个芯片对应地址码和相应的控制信号，这个地址码是由硬件电路的连接确定的。计算机通过地址总线和地址译码器找到确定的芯片，再由控制信号配合，完成对应的数据传送。
- ◇ When a computer sends data to a chip, it must simultaneously transmit the corresponding address code and control signals through the address bus. The address code is determined by the connections of hardware circuits. The computer identifies the specific chip via the address bus and address decoder, then coordinates with control signals to complete the targeted data transfer.

译码器电路的功能 Decoder Circuit Function

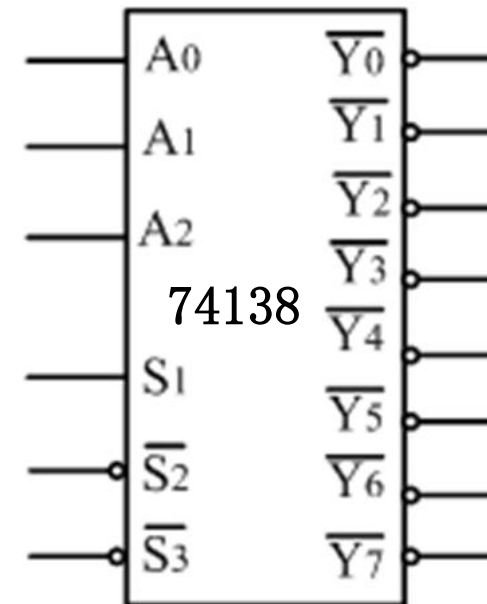
把一组输入代码的状态组合翻译成相应输出的控制电位，若输入信号有 n 个，其输出最多可以有 2^n 个。正常输出时， 2^n 个输出中仅有一个输出为低电平(或高电平)，其他 2^n-1 个输出均为高电平(或低电平)。

74138就是常用的译码器电路芯片。

The system translates input code combinations into corresponding control outputs. When there are n input signals, the output can have up to 2^n possible states. During normal operation, only one of these 2^n outputs will be at a low level (or high level), while the remaining 2^n-1 outputs will all be at a high level (or low level).

74138 is a commonly used decoder circuit chip.

输 入					输 出							
S_1	$\overline{S_2} + \overline{S_3}$	A_2	A_1	A_0	$\overline{Y_0}$	$\overline{Y_1}$	$\overline{Y_2}$	$\overline{Y_3}$	$\overline{Y_4}$	$\overline{Y_5}$	$\overline{Y_6}$	$\overline{Y_7}$
0	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0



74LS138 truth table

译码器电路的功能 Decoder Circuit Function:

74138是具有3个输入、8个输出的集成译码器，就是通常所说的三八译码器。3位二进制数字可以有8种不同的组合，或者说，3位二进制地址对应 $2^3=8$ 个不同的地址。

The 74138 is an integrated decoder with 3 inputs and 8 outputs, commonly known as a 3-to-8 decoder. A 3-bit binary address corresponds to $2^3 = 8$ different combinations.

74138译码器的作用就是将输入的三位地址状态转换为8个不同的输出之一, 分别控制8个不同电路的芯片。其中74LS138芯片是TTL电平的,74HC138是CMOS电平的。

The role of the 74138 decoder is to convert the 3-bit input address state into one of 8 different outputs, allowing it to control 8 different circuit chips. The 74LS138 is a TTL level chip, while the 74HC138 is a CMOS level chip.

使用74138与微控制器的连接如图2-11所示

The connection between 74138 and the microcontroller is shown in Figure 2-11

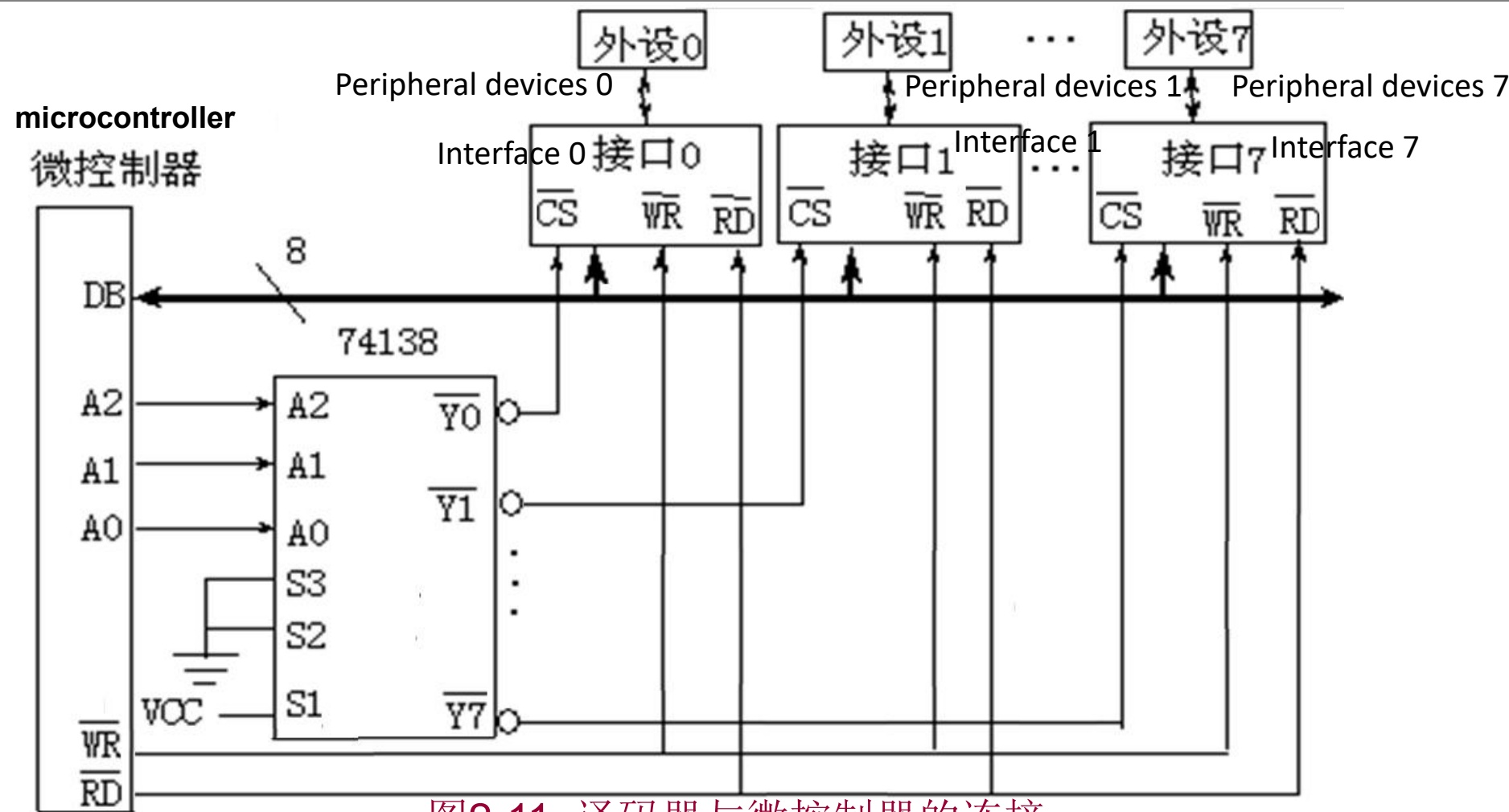


图2-11 译码器与微控制器的连接

Figure 2-11 Connection between decoder and microcontroller

74138与微控制器连接图2-11说明

Description of the Connection Diagram Between 74138 and Microcontroller (Figure 2-11)

由图2-11可知，数据线、读写控制信号线分别送到8个接口电路的相应管脚上。但是每个接口芯片的片选信号/CS接到了74138不同的输出端。74138的地址输入端接微控制器的地址总线。

As shown in Figure 2-11, the data lines and read/write control signals are routed to corresponding pins on eight interface circuits. However, each interface chips Chip Select (CS) signal connects to 74138 distinct output terminals. The address input of the 74138 is connected to the microcontrollers address bus.

根据74138的特性可知,当3位地址为000时, 74138的输出只有 $\overline{Y}_0 = 0$ (有效) $\overline{Y}_1 \sim \overline{Y}_7$ 均为高电平(无效)。

Based on the characteristics of the 74138, when the 3-bit address is 000, only the $\overline{Y}_0$ output is active (low), while outputs $\overline{Y}_1 \sim \overline{Y}_7$ are inactive (high).

所以，此时只有接口芯片0被选中，外设0可以进行接下来的读写操作，其他芯片未被选中，所以不会进行读写操作。

Therefore, at this time, only interface chip 0 is selected, and peripheral device 0 can perform the next read/write operation. Other chips are not selected, so no read/write operation will be performed.

74138与微控制器连接图2-11说明

Description of the Connection Diagram Between 74138 and Microcontroller (Figure 2-11)

尽管数据线和控制线都同时接到了每个芯片对应的管脚上，但是片选信号无效的芯片没有对应的操作，只有片选信号端有效的芯片才会有操作。

Although both the data line and control line are connected to the corresponding pin of each chip, there is no corresponding operation for the chip with invalid chip select signal, and only the chip with valid chip select signal will have operation.

由此可见，接口0、接口1、.....接口7这8个接口的地址编码分别为000、001、010、011.....111。选择不同的地址，就可以对不同的芯片进行操作，就是各个外设分时使用总线，也就完成了对不同的外设进行读写操作了。

From this, we can see that the address codes for interfaces 0, 1,... to 7 are 000,001,010,011,... 111 respectively. By selecting different addresses, we can operate on different chips. This means each peripheral device shares the bus in turn, thereby enabling read/write operations on various peripherals.

2.3 微型计算机的常用技术术语和技术

Common technical terms and techniques for microcomputers

2.3.1 常用单位及术语

Common units and terms

1、位（bit）

计算机所能表示的**最小**的数字单位，即，二进制数的位。每位只有两种状态**0**、**1**。常用**b**表示。
The **smallest unit** of a number that can be represented by a computer, namely, the bit of a binary number. Each bit has **only two states, 0 and 1**. It is often **denoted by b**.

2、字节（Byte）

8位（bit）为一个字节，是内存的基本单位，常用**B**表示。
Eight bits (bit) is a **byte**, which is the basic unit of memory. It is often denoted **by B**.

2.3.1 常用单位及术语

Common units and terms

3、字 (word)

16位二进制数称为一个字，一个字等于两个字节。

A 16-bit binary number is called a word, and a word equals two bytes.

4、字长 (Length of characters)

字长：一般说来，计算机在同一时间内处理的一组二进制数称为一个计算机的“字”，而这组二进制数的位数就是“字长”。

Word length: Generally speaking, the set of binary numbers that a computer can process at the same time is called a "word" of the computer, and the number of bits in this set of binary numbers is the "word length".

字长即字的长度, 是一次可并行处理数据的位数即数据线的条数。常与CPU内部寄存器, 运算装置, 总线宽度一致。常用微机字长有8位, 16位和32位。

Byte length refers to the number of bits in a word that can be processed in parallel, i.e., the number of data lines. It is usually consistent with the CPUs internal registers, arithmetic units, and bus width. Common microcomputer byte lengths include 8-bit, 16-bit, and 32-bit.

2.3.1 常用单位及术语

Common units and terms

5、数量单位（Quantity units）

K 千 Kilo 的符号, $1K=1024=2^{10}$

The symbol for K kilos, $1K=1024=2^{10}$

如1KB表示1024个字节

For example, 1KB represents 1024 bytes

M 兆 Mega 的符号, $1M=1K \times 1K=2^{20}$

The symbol for M Mega, $1M=1K \times 1K=2^{20}$

G 吉 Giga 的符号, $1G=1K \times 1M=2^{30}$

The symbol for Giga, $1G=1K \times 1M=2^{30}$

T 太 Tera 的符号, $1T=1M \times 1M=2^{40}$

The symbol for Tera, $1T=1M \times 1M=2^{40}$

[edit]

The BIPM specifies twenty prefixes for the International System of Units (SI).

SI prefixes

V • T • E

Prefix		Base 10	Decimal	English word		Adoption ^[nb 1]	Etymology	
Name	Symbol			Short scale	Long scale		Language	Derived word
yotta	Y	10 ²⁴	1 000 000 000 000 000 000 000 000	septillion	quadrillion	1991	Greek	eight ^[nb 2]
zetta	Z	10 ²¹	1 000 000 000 000 000 000 000	sextillion	trilliard	1991	Latin	seven ^[nb 2]
exa	E	10 ¹⁸	1 000 000 000 000 000 000	quintillion	trillion	1975	Greek	six
peta	P	10 ¹⁵	1 000 000 000 000 000	quadrillion	billiard	1975	Greek	five ^[nb 2]
tera	T	10 ¹²	1 000 000 000 000	trillion	billion	1960	Greek	four ^[nb 2] , monster
giga	G	10 ⁹	1 000 000 000	billion	milliard	1960	Greek	giant
mega	M	10 ⁶	1 000 000	million		1873	Greek	great
kilo	k	10 ³	1 000	thousand		1795	Greek	thousand
hecto	h	10 ²	100	hundred		1795	Greek	hundred
deca	da	10 ¹	10	ten		1795	Greek	ten
		10 ⁰	1	one		–		
deci	d	10 ^{−1}	0.1	tenth		1795	Latin	ten
centi	c	10 ^{−2}	0.01	hundredth		1795	Latin	hundred
milli	m	10 ^{−3}	0.001	thousandth		1795	Latin	thousand
micro	μ	10 ^{−6}	0.000 001	millionth		1873	Greek	small
nano	n	10 ^{−9}	0.000 000 001	billionth	milliardth	1960	Greek	dwarf
pico	p	10 ^{−12}	0.000 000 000 001	trillionth	billionth	1960	Spanish	peak
femto	f	10 ^{−15}	0.000 000 000 000 001	quadrillionth	billiardth	1964	Danish	fifteen, Fermi ^[nb 3]
atto	a	10 ^{−18}	0.000 000 000 000 000 001	quintillionth	trillionth	1964	Danish	eighteen
zepto	z	10 ^{−21}	0.000 000 000 000 000 000 001	sextillionth	trilliardth	1991	Latin	seven ^[nb 2]
yocto	y	10 ^{−24}	0.000 000 000 000 000 000 000 001	septillionth	quadrillionth	1991	Greek	eight ^[nb 2]

1. ^ Prefixes adopted before 1960 already existed before SI. The introduction of the **CGS system** was in 1873.
2. ^ **abcdef** Part of the beginning of the prefix was modified from the word it was derived from, ex: "peta" (prefix) vs "penta" (derived word).
3. ^ The *fermi* was introduced earlier with the same symbol "fm", in which then the "f" became a prefix. The Danish word is used since it is vaguely spelled similar to *fermi*.

2.3.1 常用单位及术语

Common units and terms

6、MIPS (Million Instructions Per Second)

MIPS为单字长定点指令平均执行速度Million Instructions Per Second的缩写，即每秒处理百万条机器语言指令数。这是衡量CPU速度的一个指标。

MIPS is short for million Instructions Per Second, the average speed at which a single-word fixed-point instruction can be executed, i.e., one million machine language instructions per second. It is a measure of CPU speed.

7、地址 (Address)

地址是微型计算机存储单元的编号，通常8bit为一个单元，每个单元有独立的编号。存储器地址的最大编号（容量）有限，由地址线的条数决定。如：

An address is the number of a microcomputer memory unit. Usually, an 8-bit unit is one unit, and each unit has an independent number. The maximum number (capacity) of a memory address is limited by the number of address lines. For example:

◆ 16条地址线的容量为64KB (0000H~FFFFH); 2^{16}

The capacity of the 16 address lines is 64KB (0000H~FFFFH); 2^{16}

◆ 20条地址线的容量1MB (00000H~FFFFFFH); 2^{20}

Capacity of 20 address lines 1MB (00000H~FFFFFFH); 2^{20}

2.3.1 常用单位及术语

Common units and terms

8、总线（Bus）

CPU是微型计算机的核心。微型计算机利用三总线将CPU与系统的其他部件如存储器、I/O接口等联系起来的。

The CPU is the core of a microcomputer. The microcomputer uses three buses to connect the CPU with other parts of the system, such as memory and I/O interfaces.

总线是具有同类性质的一组信号线。

A bus is a group of signal lines with similar properties.

三总线分别是地址总线AB(Address Bus)、数据总线DB(Data Bus)和控制总线CB(Control Bus)。

The three buses are AB (Address Bus), (Data Bus) and CB (Control Bus).

2.3.1 常用单位及术语

Common units and terms

9、访问 (Visits)

CPU对寄存器、存储器或I/O接口电路的**操作**通常分为两类：

The operation of the CPU on registers, memory, or I/O interface circuits is usually divided into two categories:

把数据存入寄存器、存储器或I/O接口电路的操作称为“**写入**”或**写操作**；

The operation of storing data into a register, memory, or I/O interface circuit is called a "**write**" or **write operation**;

把数据从寄存器、存储器或I/O接口电路取到CPU的操作称为“**读出**”或**读操作**。

The operation of taking data from a register, memory, or I/O interface circuit to the CPU is called a "**read**" or **read operation**.

这两种操作过程通常统称为“**访问**”。

These two operations are usually collectively referred to as "**access**".

2.3.1 常用单位及术语

Common units and terms

10、机器指令

Machine instructions

8051指令将一个8位二进制数据30H送到CPU的内部寄存器A

The 8051 instruction sends an 8-bit binary data 30H to the CPU internal register A.

机器指令由二进制代码(例:01110100,00110000)组成,可直接由微处理器进行译码、执行。

Machine instructions are composed of binary code (e.g., 01110100,00110000) and can be decoded and executed directly by the microprocessor.

助记符 **MOV** A, #30H
mnemonic

一条机器指令应包含要求微处理器所要完成的操作, 以及参与该操作的数据或该数据所在的地址, 有时还要有操作结果的存放地址信息;

A machine instruction should contain the operation required by the microprocessor, as well as the data involved in the operation or the address where the data is located, and sometimes the storage address information of the operation result;

这些都是以二进制数字的形式表示的, 当然,也有某些特殊指令不需要数据或地址。

These are all represented in the form of binary numbers, and of course, there are some special instructions that do not require data or addresses.

14、高级语言

High-level languages

汇编指令虽然较二进制机器指令容易阅读和编写，但还不如高级语言更接近英语自然语言。一般用高级语言编程，再用某种特殊程序翻译成机器语言。

Although assembly instructions are easier to read and write than binary machine instructions, they are not as close to English natural language as high-level languages. High-level languages are generally used for programming and then translated into machine language by a special program.

现在市面上有各种C编译器，能把C语言转换成某个具体的微处理器的机器语言。

There are various C compilers on the market that can convert C into the machine language of a specific microprocessor.

高级语言比较适宜于对汇编语言不熟悉的用户使用，其缺点是不可避免地会出现编译后的机器程序冗长、不够简练，导致程序运行时间长、速度低等问题。

High-level language is more suitable for users who are not familiar with assembly language. Its disadvantage is that the machine program after compilation will inevitably be lengthy and not concise, resulting in long running time and low speed of the program.

14、高级语言

High-level languages

另外，用汇编语言编程能更有利于硬件电路与程序的结合设计与调试。

In addition, programming in assembly language is more conducive to the combined design and debugging of hardware circuits and programs.

当然，如果用户并不在乎程序的长短和运行速度的快慢时，并且拥有对应的编译软件的前提下，采用由C语言编写，经编译程序来进行用户系统的设计和开发，也不失为一种好方法。

Of course, if the user does not care about the length of the program and the speed of running, and has the corresponding compilation software, it is also a good method to use C language to write, through the compilation program for the user system design and development.

2.3.2 Common Technologies-1. Von Neumann structure and Harvard structure

Von Neumann architecture: In 1964, Von Neumann simplified the computer structure and proposed the idea of "store program", which greatly improved the speed of computers.

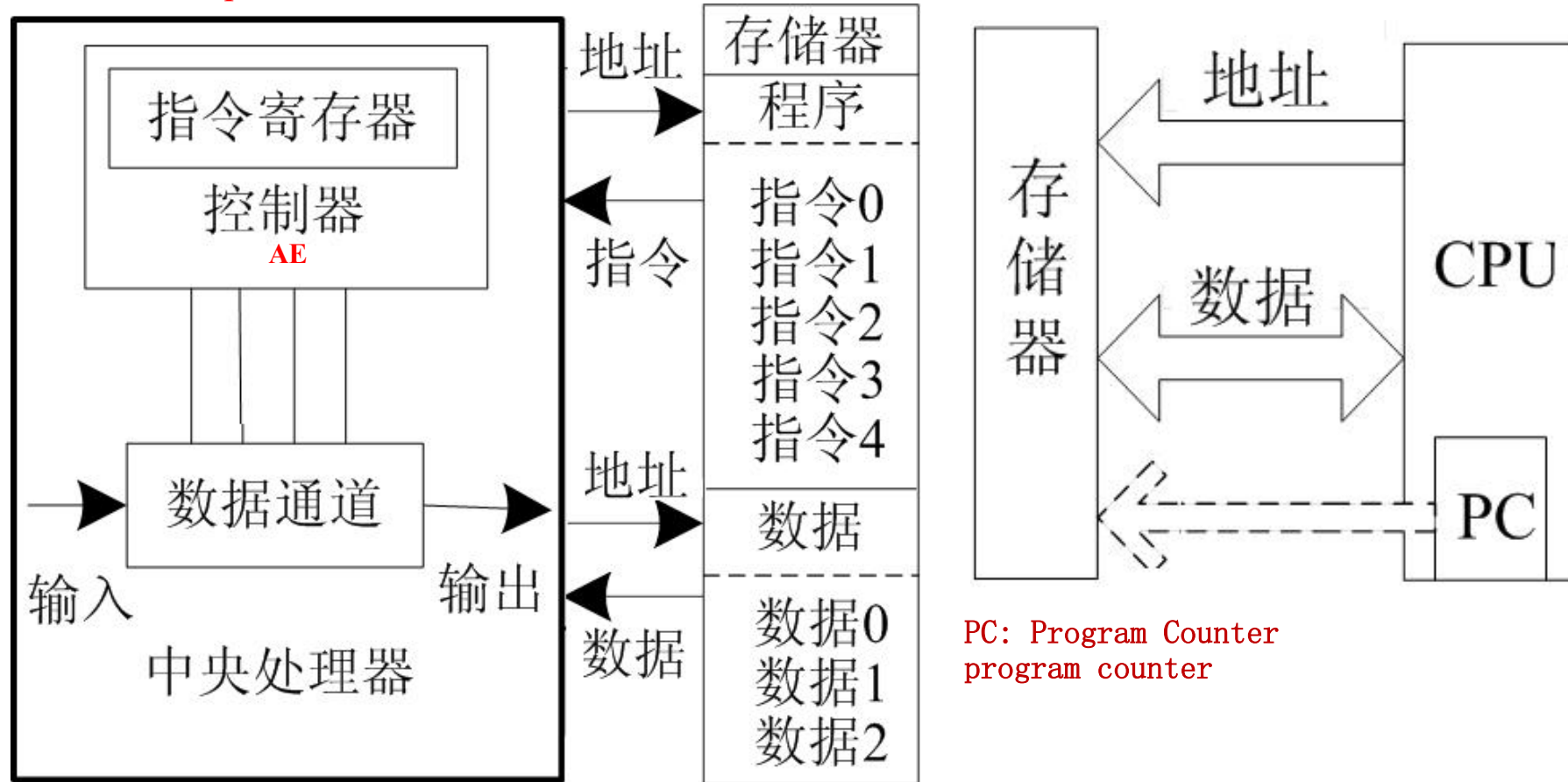


Figure 2-12 Schematic diagram of the composition of Von Neumann structure

Von Neumann structure

In the Von Neumann architecture, a computer system consists of a central processing unit (CPU) and a memory, where data and instructions are stored. The CPU can read or write to the memory based on the given address. The width of program instructions and data is the same.

Intel 8086, ARM7, MIPS processor and so on are the typical representatives of Von Neumann architecture.

MIPS means "microprocessor without interlocked piped stages" (Microprocessor without Interlocked Piped Stages)

Von Neumann structure

The idea of stored procedures can be summed up in three points:

- A computer consists of an arithmetic unit, a controller, a memory, and input/output devices.
- The computer should use binary to represent instructions and data.
- The prepared program and data are saved to the memory, and then the computer automatically takes out the instructions and data one by one for analysis, processing and execution.

Harvard Architecture

Independent system buses

In the Harvard architecture, data and programs use separate memory. The program counter (PC) only points to the program memory, not the data memory,

The consequence of this is that it is difficult to write a self-modifying program on a computer with Harvard architecture (sometimes called application programming, IAP)

➤ Advantages of Harvard architecture:

- ◆ Independent program memory and data memory provide higher performance for digital signal processing.
- ◆ Instructions and data can have different data widths and high efficiency. For example, TI's TMS320 DSP digital signal processor, Freescales MC68 series, ARM 9, ARM10 series and so on.

Harvard Architecture

Independent system buses

In the Harvard architecture, data and programs use separate memory. The program counter (PC) only points to the program memory, not the data memory,

➤ Advantages of Harvard architecture:

- ◆ Independent program memory and data memory provide higher performance for digital signal processing.
- ◆ Instructions and data can have different data widths for higher efficiency. For example, TI's TMS320 DSP digital signal processor, Freescales MC68 series, ARM 9, ARM10 series, etc.

Schematic diagram of Harvard architecture

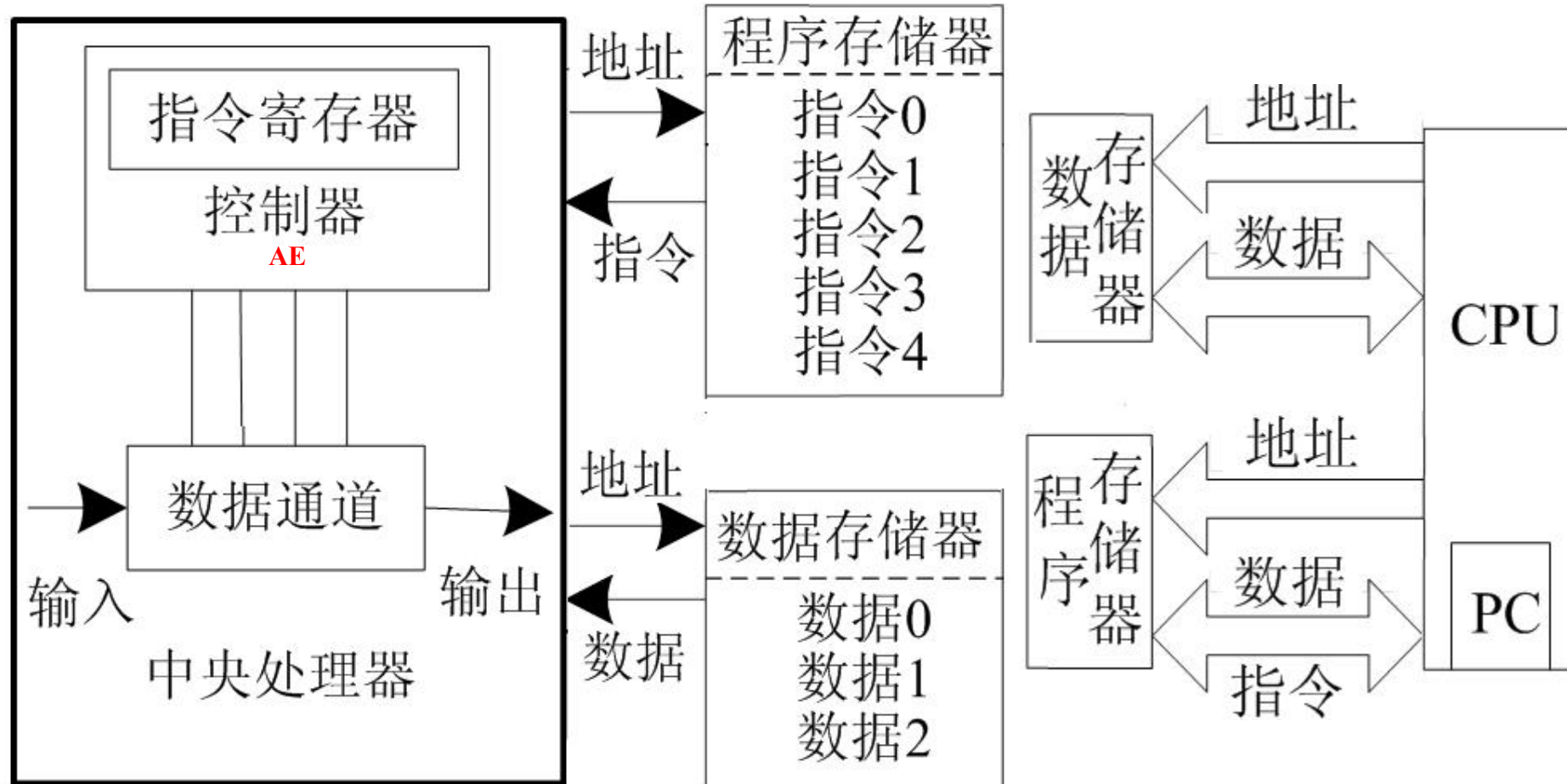


Figure 2-13 Schematic diagram of the composition of Harvard structure

2. Cache

➤ In order to solve the contradiction between fast microprocessor and slow memory, a high-speed buffer Cache is added between them.

➤ The cache uses the same semiconductor process as the CPU and matches the speed of the CPU. Its capacity accounts for about 1% of the main memory.

for instance :

◇ The cache memory of Intel80386 CPU is outside the chip, while Intel80486 CPU integrates an 8KB program and data Cache on the chip, and can be connected with a secondary cache outside the chip.

◇ The Pentium has two 8KB Cache integrated in the chip, one for program cache and one for data cache.

The role of cache memory

When the CPU reads a data from the main memory (called memory in personal computers), it first looks for the data in the Cache. If so, it reads the data from the Cache to the CPU.

Otherwise, the data is read from the main memory and sent to the CPU, while the entire data block containing the data word is sent to the Cache;

Since memory access has locality (the principle of program locality), the data to be read in several subsequent memory accesses is very likely to be located in the Cache data block that was just obtained,

With the right algorithm, the Cache hit rate can reach more than 99%, which effectively reduces the number of CPU accesses to low-speed memory, thus improving the speed of data reading and the performance of the whole machine.

3. Pipeline technology

Pipeline technology refers to a quasi-parallel processing implementation technology in which multiple instructions are operated simultaneously during program execution.

The working mode of pipeline: just like the assembly line in industrial production. In industrial manufacturing, the use of pipeline can improve the production per unit time; similarly, the use of pipeline design in CPU also helps to improve the efficiency of CPU.

The following is an example of how an assembly line works.

3. Pipeline technology

Suppose that assembling a car requires four steps:

- (1) Stamping: making body shell and chassis and other parts;
- (2) Welding: weld each part formed by stamping into the body skeleton;
- (3) Painting: cleaning, treatment, grinding, spraying and drying of body parts;
- (4) Assembly: assemble all parts into a car.

- ◇ It corresponds to four people who need to be engaged in stamping, welding, painting and final assembly.
- ◇ Assembly line manufacturing: four cars are assembled at the same time.
- ◇ If the assembly line is not used: after the first car is assembled in the above four steps, the next car starts to be assembled, and only one car is being assembled at the same time.

3. Pipeline technology

Pipeline technology can be illustrated by a space-time diagram. A space-time diagram describes the working process of a pipeline in terms of time and space. In a space-time diagram, the horizontal coordinate represents time, and the vertical coordinate represents the segments (levels) of the pipeline.

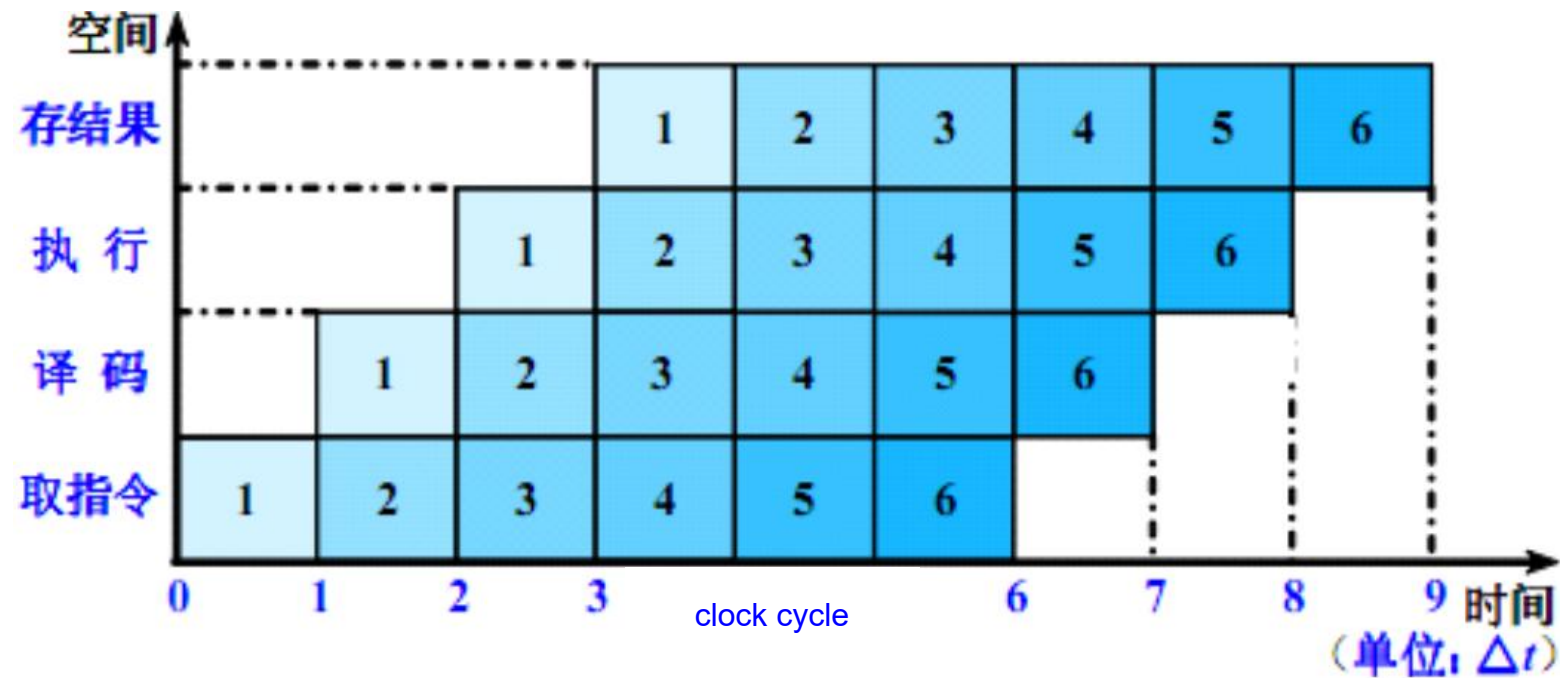


Figure 2-14 4-stage (level) instruction pipeline time-space diagram

3. Pipeline technology

The pipeline was the first time Intel used it in an Intel 486 chip. In the CPU, a 5~6 different functional circuit units form an instruction processing pipeline, and then an X86 instruction is divided into 5~6 steps and executed by these circuit units respectively, so that an instruction can be completed in one CPU clock cycle, thus improving the CPU computing speed.

In Pentium, two pipelines U and V are used. Each pipeline has its own address generation circuit, ALU and interface to connect the data Cache, so that the data can be accessed from the Cache through its own data interface.

This structure gives the Pentium a higher speed of execution, performing two integer instructions per clock cycle, twice as fast as its predecessor CPU with the same clock frequency.

4, CISC and RISC

CISC: Complex Instruction Set Computer (CISC) uses a large number of instructions, including complex instructions, to write short programs and perform complex functions.

Some complex instructions are used less frequently and cause waste of hardware and resources. CISC architecture processor has a rich instruction set, each time an instruction is executed, the processor needs to classify and look for the corresponding instruction among hundreds of instructions, so it takes some time;

- ◇ Due to the complexity of instructions, the structure complexity of the processor and the number of logical circuits are increased, the clock frequency is reduced, and the speed of instruction execution is slowed down. The pure CISC processor needs at least one more clock cycle to execute a single instruction.

4, CISC and RISC

RISC, Reduced Instruction Set Computer, short for Reduced Instruction Set Computer.

Its instruction set structure has only a few simple instructions, which simplifies the computer hardware and increases the clock frequency of the CPU. Combined with the pipeline structure, one instruction can be executed in one clock cycle, so that the performance of the whole system exceeds that of the CISC computer.

The RISC instruction set is characterized by:

- Select some of the most frequently used simple instructions;
- The instruction length is fixed, the instruction format and addressing mode are few;
- Only data and store instructions access memory, and the rest of the instructions operands are performed between registers.